KahvilaKoodi

Paperinen työpaketti

Viikot 43–7

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

Mitä rakennat ja miten etenet

KahvilaKoodi on koulun kahvilapeli, jota pelataan selaimessa. Pelaaja palvelee tiskille saapuvia asiakkaita kellon käydessä, kiire kasvaa pisteiden mukana, ja parhaat tulokset säilyvät top 5 -listalla.

1. Pelaaja avaa pelin selaimessa ilman asennusta ja aloittaa kahvilavuoron aloitusvalikosta.
2. Asiakas tilaa 1–3 tuotetta. Pelaaja valitsee tuotteet ja painaa Toimita. Peli näyttää heti, menikö toimitus oikein, ja päivittää pisteet ja jäljellä olevan ajan.
3. Kun pisteet kasvavat, kiire kasvaa: tilaukset isonevat tai asiakkaat odottavat lyhyemmän ajan. Kierros päättyy, kun aika loppuu.
4. Tulosruutu näyttää pisteet. Pelaaja tallentaa tuloksen nimimerkillä, ja viiden parhaan tuloksen lista säilyy, kun peli avataan uudelleen.

Projektin 5 vaihetta



1. Valmistelu ja suunnitelma · viikot 43–44. Selvität asiakkaan tarpeen, teet Unity-projektin ja Git-repositoryn ja kokeilet tyhjää peliä selaimessa. Suunnitelma, GitHub-issuet ja hyväksytty rajaus valmistuvat ennen koodia, jotta rakennat oikeaa peliä.

2. Pelattava peli · viikot 45–49. Rakennat pelin toiminto kerrallaan: kierros, tuotelista, kello ja pisteet, kasvava vaikeus ja top 5 -lista. Vaiheen lopussa toimeksiannon kaikki vaatimukset toimivat, joten asiakas voi kokeilla koko peliä.

3. Asiakkaan palaute ja käytettävyys · viikot 50–2. Asiakas pelaa väliversiota, ja toteutat yhdessä sovitun muutoksen omassa Git-haarassa. Sen jälkeen uusi pelaaja kokeilee peliä ilman neuvoja, ja korjaat kaksi suurinta ongelmaa.

4. Testaus ja koodin laatu · viikot 3–4. Testaat koko pelin 12 testitapauksella, korjaat kolme virhettä ketjuna ja selkeytät yhden koodikohdan katselmoinnin avulla. Laatu varmistetaan ennen julkaisuehdokasta, koska sen jälkeen peliin ei lisätä uutta.

5. Julkaisu ja näyttö · viikot 5–7. Jäädytät ominaisuudet, annat julkaisuehdokkaan RC1 asiakkaan testattavaksi ja julkaiset version v1.0 GitHub Pagesiin. Lopuksi kokoat näyttöaineiston ja harjoittelet demon.

Viikon työvaihe on tämän sivuston työohje. Kun teet muutoksen peliin, kirjaa se GitHub-issueksi ja tee se Työtapa-sivun kuudella askeleella. Valmiiseen issueen lisätään kommentti ja linkki commitiin, testien tulokset ja viikon yhteenveto kirjataan projektipäiväkirjaan.

Aikataulu

Vko	Pvm	Viikon aihe	Vaihe
43	19.–23.10.	Aloitus: peli-idea ja työkalut	1
44	26.–30.10.	Pelin suunnitelma: ruudut ja feature-lista	1
45	2.–6.11.	Feature 1 · Ensimmäinen pelattava kierros	2
46	9.–13.11.	Feature 2 · Kahvilan oikea tuotelista	2
47	16.–20.11.	Feature 3 · Kello, pisteet ja palaute	2
48	23.–27.11.	Feature 4 · Kasvava kiire	2
49	30.11.–4.12.	Feature 5 · Top 5 -tuloslista	2
50	7.–11.12.	Katselmointi · Asiakas pelaa peliäsi	3
51	14.–18.12.	Feature 6 · Asiakkaan toivoma parannus	3
52	21.–25.12.	Joululoma – ei projektityötä	–
53	28.12.–1.1.	Joululoma – ei projektityötä	–
1	4.–8.1.	Joululoma – ei projektityötä	–
2	11.–15.1.	Feature 7 · Peli ohjaa pelaajaa itse	3
3	18.–22.1.	Pelin laatu · Peli kestää pelaamista	4
4	25.–29.1.	Pelin laatu · Koodi, jonka toinen ymmärtää	4
5	1.–5.2.	Pelin julkaisu · Ensimmäinen julkaisuehdokas RC1 testipelattavana	5
6	8.–12.2.	Pelin julkaisu · v1.0 julki	5
7	15.–19.2.	Näyttö · Aineisto ja luovutus	5

Vaihe 1 – Valmistelu ja suunnitelma

Vko 43 · 19.–23.10. – Aloitus: peli-idea ja työkalut

Yhteys kokonaisprojektiin: Projekti alkaa asiakkaan toimeksiannosta, joka ei vielä kerro kaikkea: kysymyksillä selvität avoimet asiat ennen kuin rakennat mitään. Samalla teet Unity-projektin ja Git-repositoryn ja kokeilet, että tyhjä peli aukeaa selaimessa, koska koko peli julkaistaan lopuksi selainversiona. Vastaukset ja toimiva työympäristö ovat pohja viikon 44 suunnitelmalle.

Viikon tavoite: Tyhjä CafeGame-peli aukeaa selaimessa, ja tiedät asiakkaan vastauksista, millainen peli tehdään ja kenelle.

Kuvallinen tukiohje · viikko 43 · Unity 6

GitHub Desktop: kuvaohje ilman Git-komentoja

Työvaihe 1 / 5 · Lue toimeksianto ja kirjoita kuusi kysymystä asiakkaalle.

- ☐ Avaa sivun Toimeksianto-näkymä ja lue asiakkaan teksti kerran alusta loppuun.
- ☐ Kirjoita muistiin jokainen asia, jonka pelissä on pakko olla, esimerkiksi aloitusvalikko, pistelasku ja parhaiden tulosten tallennus.
- ☐ Merkitse kohdat, joita teksti ei kerro. Esimerkiksi: kuinka pitkä yksi pelikierros on?
- ☐ Kirjoita kuusi kysymystä asiakkaalle: ota mukaan Toimeksianto-näkymän Sovi asiakkaan kanssa -laatikon neljä kysymystä ja lisää kaksi omaa. Jokaisen vastauksen pitää auttaa sinua päättämään jotain pelistä.
- ☐ Kirjoita jokaisen kysymyksen perään, minkä päätöksen vastaus ratkaisee.
- ☐ Tarkista, ettei kaksi kysymystä kysy samaa asiaa. Vaihda päällekkäinen kysymys uuteen.

Valmis kun: Sinulla on kuusi erilaista kysymystä, ja jokaisen perässä lukee, minkä päätöksen vastaus ratkaisee.

Tallenna työnäyte: Kysymyslista viikon 43 päiväkirjaan, kenttään Mitä tein ja miten?

Työvaihe 2 / 5 · Pidä aloituskeskustelu ja kirjaa asiakkaan vastaukset.

- ☐ Sovi ohjaajan kanssa, kuka on asiakkaana ja milloin keskustelette.
- ☐ Kysy kysymykset yksi kerrallaan. Kirjoita vastauksen sisältö heti ylös. Nimet ja sanatarkat lausumat lähetät tarvittaessa ohjaajalle Teamsissa.
- ☐ Jos asiakas ei osaa vastata, kirjoita kohtaan sana avoin. Älä keksi vastausta itse tai tekoälyllä.
- ☐ Sovi lopuksi, mitkä ominaisuudet ovat pakollisia. Yhdessä ne ovat pakollinen perusversio (P0). Ilman niitä peliä ei voi antaa asiakkaalle.
- ☐ Kirjaa päiväkirjaan keskustelun päivä ja osallistujien roolit, esimerkiksi asiakas ja ohjaaja. Älä kirjoita muiden ihmisten nimiä.

Valmis kun: Päiväkirjassa on jokaiselle kuudelle kysymykselle joko asiakkaan vastaus tai merkintä avoin sekä lista pakollisista (P0) ominaisuuksista.

Tallenna työnäyte: Vastaukset ja avoimet asiat viikon 43 päiväkirjaan.

Työvaihe 3 / 5 · Luo Unity-projekti ja kokeile, aukeaako se selaimessa.

- ☐ Avaa Unity Hub ja valitse Installs. Tarkista, että oppilaitoksen sopima Unity-versio on asennettu ja siinä on Web Build Support.
- ☐ Jos Web Build Support puuttuu, paina version kohdalla hammasratasta, valitse Add modules ja asenna se.
- ☐ Valitse Projects → New project. Valitse pohjaksi Universal 2D, anna nimeksi CafeGame ja paina Create project.
- ☐ Tallenna scene eli pelin näkymä: File → Save As → Assets/Scenes/CafeGame.unity.
- ☐ Avaa File → Build Profiles, valitse Web ja paina Switch Platform.
- ☐ Paina Add Open Scenes ja poista rasti SampleScene-riviltä. Paina Build And Run ja valitse kansioksi Builds/Test. Unity avaa selainversion.
- ☐ Kirjoita muistiin Unity-versio. Se näkyy Unity Hubissa projektin kohdalla, esimerkiksi 6000.0.xx.

Valmis kun: Tyhjä CafeGame-peli aukeaa selaimeen, ja tiedät käyttämäsi Unity-version.

Tallenna työnäyte: Unity-versio viikon 43 päiväkirjaan. Kuvakaappaus auenneesta selainversiosta, jonka viet työvaiheessa 5 polkuun project-docs/evidence/week-43/web-test.png.

Työvaihe 4 / 5 · Luo Git-repository ja kloonaa se koneellesi.

- ☐ Asenna GitHub Desktop ja kirjaudu sisään kuvaohjeen kohtien 1–2 mukaan. Kirjoita tekijätietoihin sovittu tekijänimi ja GitHubin noreply-sähköposti.
- ☐ Luo GitHubiin uusi repository. Valitse Public, ellei ohjaaja päätä toisin: ilmaistilillä GitHub Pages toimii vain julkisessa repositoryssa.
- ☐ Valitse kohtaan Add .gitignore pohja Unity. Se pitää Unityn väliaikaiset kansiot, kuten Library ja Temp, poissa versionhallinnasta.
- ☐ Kloonaa eli kopioi repository koneellesi GitHub Desktopilla: File → Clone repository → URL. Tämä on kuvaohjeen reitti C.
- ☐ Sulje Unity. Siirrä CafeGame-projektin kansiot ja tiedostot repositoryn kansioon.
- ☐ Lisää projekti uudesta paikasta Unity Hubiin: Add → Add project from disk. Avaa projekti ja tarkista, että CafeGame-scene aukeaa.

Valmis kun: Repository on GitHubissa, ja Unity Hub avaa CafeGame-projektin repositoryn kansiosta.

Tallenna työnäyte: Repositoryn linkki viikon 43 päiväkirjaan.

Työvaihe 5 / 5 · Tee ensimmäinen commit ja push.

- ☐ Luo repositoryn juureen kansio project-docs ja sen sisään tyhjä tiedosto projektipaivakirja.md. Git ei tallenna tyhjää kansiota.
- ☐ Siirrä työvaiheen 3 kuvakaappaus polkuun project-docs/evidence/week-43/web-test.png.
- ☐ Kirjoita README.md-tiedostoon pelin nimi ja yksi virke siitä, mitä peli tekee. README on repositoryn esittelytiedosto.
- ☐ Tarkista GitHub Desktopin Changes-listasta, että Assets, Packages, ProjectSettings, project-docs ja README.md ovat mukana. Library, Temp ja Builds eivät saa olla mukana.
- ☐ Tee commit eli tallenna muutokset versionhallintaan: kirjoita GitHub Desktopin Summary-kenttään ”Unity-projektin pohja” ja paina Commit to main. Tee sitten push painamalla Push origin, jolloin commit siirtyy GitHubiin.

Valmis kun: Commit näkyy GitHubissa. Repositoryssa ovat Assets-, Packages-, ProjectSettings- ja project-docs-kansiot sekä README.md.

Tallenna työnäyte: Repositoryn linkki ja ensimmäisen commitin tunniste (7 merkin koodi GitHubin commit-listassa, esimerkiksi a1b2c3d) viikon 43 päiväkirjaan.

Viikon lopputarkistus: Asiakkaan vastaukset ja avoimet asiat ovat päiväkirjassa, tyhjä peli aukeaa selaimessa ja ensimmäinen commit näkyy GitHubissa.

Työnäyte Git-repositoryyn ennen rastia: kysymykset ja asiakkaan vastaukset päiväkirjassa, kuvakaappaus selainversiosta, repository-linkki ja ensimmäinen commit.

Vko 44 · 26.–30.10. – Pelin suunnitelma: ruudut ja feature-lista

Yhteys kokonaisprojektiin: Viikon 43 asiakasvastausten perusteella muutat toimeksiannon suunnitelmaksi: pelin säännöt GDD:hen, pienet GitHub-issuet, ruutujen luonnos ja skriptien kartta. Rajaus estää peliä kasvamasta liian suureksi, ja Valmis kun -ehdoista näet, milloin työn voi testata. Viikosta 45 alkaen rakennat ruudut luonnoksen ja skriptit kartan mukaan.

Viikon tavoite: Peli on suunniteltu ennen koodia: GDD on repositoryssa, GitHub-issuet on priorisoitu ja ohjaaja on hyväksynyt rajauksen.

Kuvallinen tukiohje · viikko 44 · Unity 6

Täytä GDD tällä sivulla

Avaa koko toimeksianto

Työvaihe 1 / 5 · Täytä pelin suunnitteludokumentti (GDD) ja vie se repositoryyn.

- □ Avaa Suunnitelma (GDD) -näkymä. GDD eli Game Design Document on pelin suunnitteludokumentti. Lue esitäytetty osa: se tulee toimeksiannosta.
- □ Kirjoita pelille työnimi ja oma nimesi.
- □ Kirjoita kahdella tai kolmella virkkeellä, mitä peli tavoittelee, kenelle se on ja mikä on sinun roolisi.
- □ Valitse visuaalinen tyyli. Kirjoita, mistä grafiikka tulee ja millä lisenssillä eli käyttöluvalla sitä saa käyttää, esimerkiksi Kenney.nl, CC0 (saa käyttää vapaasti).
- □ Päätä pisteet: montako pistettä pelaaja saa oikeasta toimituksesta ja montako hän menettää väärästä. Perustele valinta yhdellä tai kahdella virkkeellä.
- □ Jos sovit kierroksen pituuden asiakkaan kanssa viikolla 43, kirjaa se. Muuten jätä kenttä tyhjäksi: se on avoin asia.
- □ Paina Lataa gdd.md. Siirrä tiedosto repositoryn project-docs-kansioon, tee commit ja push.

Valmis kun: Tiedosto project-docs/gdd.md näkyy GitHubissa, eikä yhdessäkään OMA-merkityssä kentässä ole tyhjää kohtaa. ASIAKAS-merkitty kierroksen pituus saa jäädä avoimeksi.

Tallenna työnäyte: project-docs/gdd.md ja commit-linkki viikon 44 päiväkirjaan.

Työvaihe 2 / 5 · Pilko ensimmäiset featuret pieniksi GitHub-issueiksi.

- □ Avaa Suunnitelma-näkymän lista Featuret tekojärjestyksessä. Feature on yksi pelaajalle näkyvä ominaisuus, esimerkiksi pistelasku.
- □ Ota listan kaksi ensimmäistä featurea. Kirjoita jokaisesta paperille 3–5 pientä asiaa, joita sen tekemiseen tarvitaan, esimerkiksi pelattava kierros: kolme ruutua, ruudun vaihto, tilaus ja pisteet.
- □ Avaa GitHubissa repository → Issues → New issue. Tee jokaisesta pienestä asiasta oma issue eli tehtävä. Aloita otsikko verbillä, esimerkiksi Näytä asiakkaan tilaus.
- □ Kirjoita jokaiseen issueen arvio: puoli päivää tai yksi päivä. Jos tehtävä on isompi, jaa se kahdeksi issueksi.
- □ Kirjoita jokaiseen issueen Valmis kun -ehto: mitä toinen ihminen näkee pelissä, kun tehtävä on tehty.
- □ Lisää vielä yksi issue kustakin muusta featuresta paitsi asiakkaan toivomasta parannuksesta, joka selviää vasta viikolla 50. Pilkot ne tarkemmin sillä viikolla, kun teet featuren.

Valmis kun: GitHubissa on issuet kahdelle ensimmäiselle featurelle ja yksi issue kustakin muusta. Jokaisessa on arvio ja Valmis kun -ehto.

Tallenna työnäyte: Linkki GitHubin Issues-listaan viikon 44 päiväkirjaan.

Työvaihe 3 / 5 · Merkitse jokaiselle issuelle, onko se pakollinen, tärkeä vai lisä.

- □ Luo GitHubissa kolme labelia eli tunnistetta: Issues → Labels → New label. Anna nimiksi P0 pakollinen, P1 tärkeä ja P2 lisä.
- □ Anna label P0 pakollinen jokaiselle issuelle, jota ilman peliä ei voi antaa asiakkaalle. Kaikki toimeksiannon vaatimukset ovat pakollisia (P0). Käytä apuna viikon 43 aloituskeskustelun pakollisten (P0) listaa.
- □ Anna label P1 tärkeä issueille, jotka parantavat peliä. Peli on kuitenkin valmis ilman niitä. Tärkeät (P1) tehdään vasta, kun pakolliset (P0) toimivat.
- □ Anna label P2 lisä issueille, jotka voi jättää pois, jos aika loppuu. Lisät (P2) tehdään viimeisenä.
- □ Näytä lista ohjaajalle ja pyydä hyväksyntä rajaukselle eli sille, mitä peliin tehdään ja mitä jätetään pois. Kirjaa päiväkirjaan hyväksyjän rooli ja päivä.

Valmis kun: Jokaisella issuella on yksi label: P0 pakollinen, P1 tärkeä tai P2 lisä. Ohjaaja on hyväksynyt listan.

Tallenna työnäyte: Linkki Issues-listaan sekä hyväksyjän rooli ja päivä viikon 44 päiväkirjaan.

Työvaihe 4 / 5 · Piirrä pelin kolme ruutua paperille.

- □ Ota paperi ja jaa se kolmeen osaan. Kirjoita osien otsikoiksi Valikko, Peli ja Tulos.
- □ Piirrä Valikko-ruutuun pelin nimi ja Aloita-painike.
- □ Piirrä Peli-ruutuun asiakkaan tilaus, tuotepainikkeet (esimerkiksi Kahvi, Tee ja Sämpylä), Toimita-painike, aika ja pisteet.
- □ Piirrä Peli-ruutuun myös kohta, jossa peli kertoo, menikö toimitus oikein.
- □ Piirrä Tulos-ruutuun loppupisteet, viiden parhaan tuloksen lista ja Pelaa uudelleen -painike.
- □ Piirrä nuoli jokaisesta ruutua vaihtavasta painikkeesta siihen ruutuun, johon se vie, ja Peli-ruudusta Tulos-ruutuun, kun aika loppuu. Kuvaa paperi ja tallenna kuva nimellä mockup.png eli käyttöliittymän luonnoskuva.

Valmis kun: Kuvassa on kolme ruutua, ja nuolet näyttävät, mihin Aloita ja Pelaa uudelleen vievät ja milloin peli siirtyy tulosruutuun.

Tallenna työnäyte: Kuva polkuun project-docs/evidence/week-44/mockup.png, commit ja push.

Työvaihe 5 / 5 · Piirrä kartta pelin C#-skripteistä ja niiden tehtävistä.

- □ Piirrä paperille viisi laatikkoa: GameManager, OrderManager, ProductDatabase, DifficultyController ja SaveService. Jokainen laatikko on yksi C#-skripti eli ohjelmatiedosto.
- □ Kirjoita jokaisen laatikon alle omin sanoin, mitä skripti tekee kahvilapelissä. Apuna on lista kortin lopun suljetussa kohdassa Skriptien tehtävät.
- □ Piirrä nuolet: products.json → ProductDatabase, ProductDatabase → OrderManager (tuotelista) ja SaveService → selaimen tallennus. Tuotteet tulevat tiedostosta, ja tulokset säilyvät selaimessa.
- □ Kuvaa paperi ja tallenna kuva nimellä unity-rakenne.png. Näytä kuva ohjaajalle ennen koodaamista ja kirjaa päiväkirjaan hänen kommenttinsa, roolinsa ja päivä.

Valmis kun: Kuvassa on viisi skriptiä, jokaisella oma tehtävä, ja nuolet näyttävät, mistä tuotteet ja tulokset kulkevat.

Tallenna työnäyte: Kuva polkuun project-docs/evidence/week-44/unity-rakenne.png, commit ja push.

Viikon lopputarkistus: Tiedosto gdd.md on repositoryssa, jokaisella issuella on arvio, tärkeysluokka – pakollinen (P0), tärkeä (P1) tai lisä (P2) – ja Valmis kun -ehto, ja ohjaaja on hyväksynyt rajauksen. Luonnoksessa näkyvät valikko, peli ja tulos.

Työnäyte Git-repositoryyn ennen rastia: gdd.md, Issues-lista prioriteetteineen, rajauksen hyväksyntä, mockup.png ja unity-rakenne.png.

Vaihe 2 – Pelattava peli

Vko 45 · 2.–6.11. – Feature 1 · Ensimmäinen pelattava kierros

Yhteys kokonaisprojektiin: Viikon 44 luonnoksesta ja skriptien kartasta tulee nyt ensimmäinen pelattava versio. Rakennat kolme ruutua ja GameManagerin, ja yksi kiinteä kahvitilaus riittää, koska pieni alusta loppuun toimiva peli paljastaa Canvasin ja painikkeiden kytkentävirheet aikaisin. Viikolla 46 kiinteä kahvi vaihtuu tuotelistasta arvottuun tilaukseen.

Viikon tavoite: Peliä voi pelata alusta loppuun: Aloita → asiakas tilaa kahvin → pelaaja toimittaa → tulosruutu näyttää pisteet.

Kuvallinen tukiohje · viikko 45 · Unity 6

Kenney.nl – ilmaiset CC0-assetit: hahmot, esineet ja käyttöliittymäkuvat

OpenGameArt – 2D-hahmot ja taustat (tarkista lisenssi)

Piskel – piirrä omat spritet selaimessa

Työvaihe 1 / 5 · Rakenna Unityyn valikkoruutu.

- ☐ Avaa CafeGame-scene. Valitse Hierarchy-ikkunassa eli scenen objektilistassa + → UI → Canvas. Canvas on alue, jolle pelin tekstit ja painikkeet tulevat.
- ☐ Klikkaa Canvasia hiiren oikealla ja valitse UI → Panel. Tee näin kolme paneelia ja nimeä ne MenuPanel, GamePanel ja ResultPanel.
- ☐ Lisää MenuPaneliin painike: UI → Button - TextMeshPro. Paina Import TMP Essentials, jos Unity kysyy. Nimeä painike StartButton.
- ☐ Valitse StartButtonin alta Text (TMP) ja kirjoita Inspectorin tekstikenttään Aloita.
- ☐ Lisää MenuPaneliin teksti (UI → Text - TextMeshPro) ja kirjoita siihen pelin nimi.
- ☐ Siirrä Scene-ikkunassa pelin nimi ja Aloita-painike luonnoksesi (mockup.png) mukaisille paikoille.

Valmis kun: Hierarchyssä on Canvas ja sen alla kolme paneelia. MenuPanelissa näkyvät pelin nimi ja Aloita-painike.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 45 päiväkirjaan.

Työvaihe 2 / 5 · Rakenna peli- ja tulosruutu.

- ☐ Lisää GamePaneliin kolme tekstiä (UI → Text - TextMeshPro): OrderText, ScoreText ja TimeText.
- ☐ Lisää GamePaneliin kaksi painiketta: CoffeeButton tekstillä Kahvi ja SubmitButton tekstillä Toimita.
- ☐ Lisää ResultPaneliin teksti FinalScoreText ja painike RestartButton tekstillä Pelaa uudelleen.
- ☐ Siirrä Scene-ikkunassa jokaisen paneelin tekstit ja painikkeet erilleen luonnoksesi (mockup.png) mukaan, jotta ne eivät ole päällekkäin.
- ☐ Piilota GamePanel ja ResultPanel: valitse paneeli ja poista rasti Inspectorin yläreunasta nimen vierestä.

Valmis kun: Hierarchy näyttää samalta kuin avun rakenne (Main Camera, Global Light 2D ja EventSystem saavat olla lisäksi), ja Play-tilassa näkyy vain valikko.

Tallenna työnäyte: Kuvakaappaus Hierarchy-ikkunasta polkuun project-docs/evidence/week-45/hierarchy.png, commit ja push. Polku viikon 45 päiväkirjan kenttään Missä työnäyte on?

Työvaihe 3 / 5 · Tee GameManager-skripti, joka vaihtaa ruutua.

- ☐ Luo Project-ikkunan Assets-kansioon kansio Scripts: hiiren oikea → Create → Folder.
- ☐ Klikkaa Scripts-kansiota hiiren oikealla ja valitse Create → Scripting → MonoBehaviour Script eli objektiin liitettävä C#-skripti. Anna nimeksi GameManager.
- ☐ Avaa skripti ja korvaa sen sisältö avun työpohjalla. Tallenna tiedosto.
- ☐ Luo Hierarchyyn tyhjä objekti: + → Create Empty. Nimeä se GameManager ja raahaa GameManager-skripti sen päälle.
- ☐ Valitse GameManager-objekti. Raahaa MenuPanel, GamePanel ja ResultPanel Hierarchystä Inspectorin kenttiin Menu Panel, Game Panel ja Result Panel.
- ☐ Kytke Aloita-painike. Valitse StartButton ja paina On Click () -listan +. Raahaa GameManager-objekti kenttään ja valitse GameManager → StartGame.
- ☐ Kytke RestartButton samalla tavalla metodiin eli skriptin toimintoon RestartGame.
- ☐ Paina Play ja sitten Aloita. Peliruutu tulee näkyviin, ja valikko katoaa.

Valmis kun: Play-tilassa Aloita vaihtaa valikon peliruutuun.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 45 päiväkirjaan.

Työvaihe 4 / 5 · Tee ensimmäinen tilaus: asiakas tilaa kahvin ja pelaaja toimittaa sen.

- ☐ Valitse GameManager-objekti. Raahaa Inspectorissa OrderText, ScoreText ja FinalScoreText skriptin kenttiin.
- ☐ Täydennä StartGame-metodin TODO-rivit eli koodiin merkityt täydennettävät kohdat: kirjoita OrderText-tekstiin ”Asiakas tilaa: Kahvi” ja näytä pisteet ScoreText-tekstissä.
- ☐ Kytke CoffeeButtonin On Click () -listaan GameManager → SelectCoffee. Kytke SubmitButtonin listaan GameManager → SubmitOrder.
- ☐ Täydennä SubmitOrder: jos kahvi on valittu, lisää pisteisiin 10 ja päivitä ScoreText.
- ☐ Täydennä EndGame: näytä pisteet FinalScoreText-kentässä.
- ☐ Paina Play ja pelaa: Aloita → Kahvi → Toimita.

Valmis kun: Aloita → Kahvi → Toimita → tulosruudulla lukee 10 pistettä.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 45 päiväkirjaan.

Työvaihe 5 / 5 · Testaa kierros kolme kertaa selaimessa ja kirjaa tulokset.

- ☐ Tee selainversio: File → Build Profiles → Web → Build And Run.
- ☐ Kirjoita päiväkirjaan ennen pelaamista odotettu tulos: Aloita → Kahvi → Toimita → tulosruudulla 10.
- ☐ Pelaa kierros kolme kertaa. Käytä välillä Pelaa uudelleen -painiketta.
- ☐ Kirjaa jokaisesta kierroksesta, menikö se odotetusti. Jos ei mennyt, kirjoita tarkasti, mitä tapahtui.
- ☐ Korjaa löytämäsi virheet vasta kirjaamisen jälkeen. Kirjaa korjaus samaan kohtaan.
- ☐ Tee commit ja push. Kirjoita commit-viestiin, mitä korjasit.
- ☐ Kirjoita GitHubissa jokaiseen tämän viikon valmiiseen issueen kommentti. Lisää kommenttiin linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Päiväkirjassa on kolme testikierrosta tuloksineen, ja viimeinen kierros meni odotetusti.

Tallenna työnäyte: Kolme kuvaa tai lyhyt video pelipolusta kansioon project-docs/evidence/week-45/, commit ja push. Testikirjaukset ja commit-linkki viikon 45 päiväkirjaan.

Viikon lopputarkistus: Aloita → Kahvi → Toimita → tulosruutu näyttää pisteet. Polku toimii selainversiossa ilman, että kosket Unity-editoriin kesken pelin.

Työnäyte Git-repositoryyn ennen rastia: video tai kuvat pelipolusta, kolme testikierrosta päiväkirjassa ja commit-linkit.

Vko 46 · 9.–13.11. – Feature 2 · Kahvilan oikea tuotelista

Yhteys kokonaisprojektiin: Viikon 45 kierros toimii, mutta kahvi on kirjoitettu suoraan koodiin. Nyt tuotteet luetaan erillisestä products.json-tiedostosta, kuten toimeksianto vaatii, ja OrderManager arpoo tilauksen ja tarkistaa toimituksen. Tilaus ja sen pisteet ovat pohja viikon 47 kellolle, palautteelle ja pelisäännöille.

Viikon tavoite: Asiakas tilaa 1–3 tuotetta, jotka arvotaan products.json-tiedostosta, ja tuotteita voi muuttaa koskematta C#-koodiin.

Työvaihe 1 / 5 · Kirjoita kahvilan tuotteet products.json-tiedostoon.

- ☐ Luo Assets-kansioon kansio Data. Luo sinne koodieditorilla, esimerkiksi VS Codella, tiedosto products.json.
- ☐ Kirjoita tiedostoon kolme tuotetta avun mallin mukaan. Jokaisella tuotteella on id (tunniste), name (nimi) ja points (pisteet).
- ☐ Anna points-arvoksi se pistemäärä, jonka päätit pelin suunnitteludokumentissa (GDD) oikealle toimitukselle, esimerkiksi 10.
- ☐ Tarkista tiedoston muoto: lainausmerkit nimien ympärillä, pilkut tuotteiden välissä ja hakasulkeet listan ympärillä. VS Code näyttää muotovirheen punaisella.
- ☐ Palaa Unityyn ja klikkaa products.json-tiedostoa Project-ikkunassa. Inspector näyttää tiedoston sisällön.

Valmis kun: Unityn Inspector näyttää products.json-tiedoston, ja siinä on kolme tuotetta.

Tallenna työnäyte: Commit ja push. Tiedoston polku viikon 46 päiväkirjaan.

Työvaihe 2 / 5 · Lue tuotteet tiedostosta ProductDatabase-skriptillä.

- ☐ Luo Scripts-kansioon skripti ProductDatabase ja korvaa sen sisältö avun työpohjalla.
- ☐ Lue työpohjasta kaksi luokkaa: ProductData on yksi tuote ja ProductList on tuotteiden lista. Rivi [System.Serializable] kertoo Unitylle, että luokan voi lukea JSONista.
- ☐ Luo Hierarchyyn tyhjä objekti ProductDatabase ja raahaa skripti sen päälle.
- ☐ Raahaa products.json Inspectorissa ProductDatabase-skriptin Products Json -kenttään.
- ☐ Täydennä ensimmäinen TODO-kohta: jos Products Json -kenttä on tyhjä, kirjoita virheilmoitus Consoleen eli Unityn viesti-ikkunaan ja palauta tyhjä lista.
- ☐ Täydennä toinen TODO-kohta: ota rikkinäisen JSONin virhe kiinni try–catch-lohkolla eli koodilla, joka nappaa virheen ennen kuin peli kaatuu. Kirjoita virheilmoitus Consoleen ja palauta tyhjä lista.

Valmis kun: Avun tarkistustestissä Console näyttää Kahvi, ja molemmat TODO-kohdat on täydennetty.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 46 päiväkirjaan.

Työvaihe 3 / 5 · Arvo asiakkaalle 1–3 tuotteen tilaus OrderManager-skriptillä.

- ☐ Luo skripti OrderManager avun työpohjasta ja liitä se GameManager-objektiin.
- ☐ Raahaa ProductDatabase-objekti OrderManager-skriptin Product Database -kenttään.
- ☐ Täydennä CreateOrder: arvo tilaukseen 1–3 tuotetta listasta ja palauta tilausteksti, esimerkiksi ”Asiakas tilaa: Kahvi, Sämpylä”.
- ☐ Lisää GameManageriin kenttä [SerializeField] private OrderManager orderManager; ja raahaa GameManager-objekti Inspectorissa sen Order Manager -kenttään.
- ☐ Muuta StartGame-metodia: kirjoita OrderText-tekstiin CreateOrder-metodin palauttama tilausteksti.
- ☐ Paina Play ja Aloita viisi kertaa. Kirjaa, mitä tilauksia tuli.

Valmis kun: Play-tilassa tilaus vaihtuu joka pelissä, ja siinä on 1–3 tuotetta.

Tallenna työnäyte: Commit ja push. Commit-linkki ja viiden tilauksen lista viikon 46 päiväkirjaan.

Työvaihe 4 / 5 · Tee tuotepainikkeet ja tarkista, menikö toimitus oikein.

- ☐ Lisää GamePaneliin painikkeet TeaButton tekstillä Tee ja BreadButton tekstillä Sämpylä.
- ☐ Poista Kahvi-painikkeen vanha kytkentä: valitse CoffeeButton, valitse On Click () -listan SelectCoffee-rivi ja paina –.
- ☐ Kytke kolme tuotepainiketta GameManager-objektin OrderManager → SelectProduct -metodiin. Kirjoita parametriksi eli annettavaksi arvoksi tuotteen id: kahvi, tee tai sampyla.
- ☐ Täydennä CheckDelivery: vertaa valittuja tuotteita tilaukseen. Oikeasta toimituksesta metodi palauttaa tuotteiden points-arvojen summan. Väärästä toimituksesta se palauttaa nollan.
- ☐ Muuta SubmitOrder-metodia: lisää pisteisiin CheckDelivery-metodin palauttama arvo.
- ☐ Paina Play ja toimita yksi oikea ja yksi väärä tilaus. Kirjaa, montako pistettä kumpikin antoi.

Valmis kun: Oikea toimitus antaa tuotteiden pisteet yhteensä, ja väärä toimitus antaa 0 pistettä.

Tallenna työnäyte: Commit ja push. Commit-linkki ja kahden toimituksen tulos viikon 46 päiväkirjaan.

Työvaihe 5 / 5 · Testaa tuotelista kolmella testillä.

- ☐ Kirjoita päiväkirjaan odotettu tulos kolmelle testille ennen kuin testaat.
- ☐ Testi 1: muuta kahvin points-arvo 10 → 15 ja paina Play. Toimita oikein tilaus, jossa on kahvi. Pisteet ovat tuotteiden summa, jossa kahvi on 15, esimerkiksi Kahvi + Tee = 25.
- ☐ Testi 2: tyhjennä ProductDatabase-skriptin Products Json -kenttä ja paina Play. Consolessa näkyy virheilmoitus, eikä peli kaadu.
- ☐ Testi 3: poista products.json-tiedostosta yksi pilkku ja paina Play. Consolessa näkyy virheilmoitus, eikä peli kaadu.
- ☐ Palauta tiedosto ja kenttä ennalleen. Kirjaa jokaisen testin todellinen tulos päiväkirjaan.
- ☐ Kirjoita päiväkirjaan omin sanoin, mitä kukin vaihe tekee tiedon kulussa: products.json → ProductDatabase → OrderManager → GameManager → OrderText.
- ☐ Kirjoita GitHubissa jokaiseen tämän viikon valmiiseen issueen kommentti. Lisää kommenttiin linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Kaikki kolme testiä menivät odotetusti, ja tulokset ovat päiväkirjassa.

Tallenna työnäyte: Testien tulokset ja tiedon kulku viikon 46 päiväkirjaan.

Viikon lopputarkistus: Kun muutat kahvin pistearvon products.json-tiedostossa, muutos näkyy pelissä ilman C#-muutosta. Jos tiedosto puuttuu tai on rikki, peli näyttää virheilmoituksen eikä kaadu.

Työnäyte Git-repositoryyn ennen rastia: products.json, ProductDatabase.cs ja OrderManager.cs committeina sekä kolmen testin tulokset.

Vko 47 · 16.–20.11. – Feature 3 · Kello, pisteet ja palaute

Yhteys kokonaisprojektiin: Viikon 46 tilaus ja toimituksen tarkistus antavat pisteet, mutta peli päättyy vielä ensimmäiseen toimitukseen. Kirjoitat ensin pelisäännöt GDD:n pisteytyksen pohjalta ja lisäät sitten ajastimen, palautteen ja rajatestit, jotta pelaaja voi tehdä päätöksiä ajan ja tuloksen perusteella. Kun kierros kestää ajan loppuun, viikolla 48 kiirettä voi kasvattaa.

Viikon tavoite: Kello laskee, pelaaja näkee heti, menikö toimitus oikein, ja kierros päättyy kerran, kun aika loppuu.

Työvaihe 1 / 4 · Kirjoita pelin säännöt ennen koodaamista.

- ☐ Avaa project-docs/gdd.md. Katso, mitkä pisteet päätit oikealle ja väärälle toimitukselle.
- ☐ Kirjoita päiväkirjaan sääntö 1: oikea toimitus antaa tuotteiden pisteet yhteensä.
- ☐ Kirjoita sääntö 2: väärä toimitus vie pois sen määrän pisteitä, jonka päätit pelin suunnitteludokumentissa (GDD).
- ☐ Kirjoita sääntö 3: kierros kestää __ sekuntia. Käytä asiakkaan kanssa sovittua pituutta. Jos sitä ei ole sovittu, käytä 60 sekuntia ja merkitse asia avoimeksi.
- ☐ Kirjoita sääntö 4: peli päättyy kerran, kun aika on 0. Kirjoita sääntö 5: voivatko pisteet mennä alle nollan, ja perustele oma päätöksesi yhdellä virkkeellä.

Valmis kun: Päiväkirjassa on viisi numeroitua sääntöä lukuarvoineen.

Tallenna työnäyte: Säännöt viikon 47 päiväkirjaan.

Työvaihe 2 / 4 · Lisää peliin ajastin, joka päättää kierroksen.

- ☐ Lisää GameManageriin avun työpohjan neljä kenttää: timeText, roundSeconds (kierroksen pituus), timeLeft (jäljellä oleva aika) ja isPlaying (onko peli käynnissä).
- ☐ Raahaa TimeText Inspectorissa GameManager-skriptin Time Text -kenttään ja kirjoita Round Seconds -kenttään säännön 3 kierroksen pituus.
- ☐ Täydennä StartGame: timeLeft saa kierroksen pituuden ja isPlaying arvon true.
- ☐ Lisää avun Update-metodi GameManageriin. Se vähentää aikaa Time.deltaTime-arvolla eli edellisestä ruudunpäivityksestä kuluneella ajalla. Täydennä sen TODO: näytä aika TimeText-tekstissä kokonaisina sekunteina.
- ☐ Kirjoita EndGame-metodin ensimmäiseksi riviksi isPlaying = false, jotta Update ei kutsu EndGamea uudelleen eikä peli pääty kahdesti.
- ☐ Poista SubmitOrder-metodista viikolla 45 lisätty EndGame-kutsu. Nyt kierros päättyy vasta, kun aika loppuu.

Valmis kun: Play-tilassa aika laskee, pelaaja voi toimittaa monta tilausta ja tulosruutu aukeaa kerran, kun aika on 0.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 47 päiväkirjaan.

Työvaihe 3 / 4 · Näytä pelaajalle heti, menikö toimitus oikein.

- ☐ Lisää GamePaneliin teksti FeedbackText: UI → Text - TextMeshPro.
- ☐ Lisää GameManageriin kenttä [SerializeField] private TMP_Text feedbackText; ja raahaa FeedbackText Inspectorissa Feedback Text -kenttään.
- ☐ Toimitus on oikein, kun CheckDelivery palauttaa enemmän kuin 0. Silloin lisää pisteet ja kirjoita FeedbackText-tekstiin esimerkiksi ”Oikein! +10”.
- ☐ Kun toimitus on väärin, vähennä pisteitä säännön 2 mukaan, noudata sääntöä 5 (alle nollan vai ei) ja kirjoita esimerkiksi ”Väärä tilaus –5”.
- ☐ Päivitä ScoreText heti jokaisen toimituksen jälkeen.
- ☐ Tee toimituksen jälkeen uusi tilaus CreateOrder-metodilla, jotta peli jatkuu, kunnes aika loppuu.

Valmis kun: Oikea ja väärä toimitus näyttävät eri palautteen, ja pisteet muuttuvat sääntöjen mukaan.

Tallenna työnäyte: Kuvakaappaukset molemmista palautteista kansioon project-docs/evidence/week-47/, commit ja push. Polut viikon 47 päiväkirjaan.

Työvaihe 4 / 4 · Testaa pelin kolme rajatilannetta.

- ☐ Kirjoita päiväkirjaan jokaiselle kolmelle testille odotettu tulos ennen kuin testaat.
- ☐ Testi 1: aseta Inspectorin Round Seconds -arvoksi 3, paina Play ja Aloita ja odota. Tulosruutu aukeaa vain kerran.
- ☐ Testi 2: paina Toimita kaksi kertaa nopeasti peräkkäin. Vain ensimmäinen painallus lasketaan.
- ☐ Testi 3: paina tulosruudulla Pelaa uudelleen. Aika ja pisteet alkavat alusta.
- ☐ Jos testi ei mennyt odotetusti, korjaa vika, tee commit ja testaa uudelleen. Kirjaa molemmat ajot.
- ☐ Kirjoita GitHubissa jokaiseen tämän viikon valmiiseen issueen kommentti. Lisää kommenttiin linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Kolmen rajatestin odotetut ja todelliset tulokset on kirjattu, ja viimeinen ajo meni odotetusti.

Tallenna työnäyte: Testien tulokset ja korjausten commit-linkit viikon 47 päiväkirjaan.

Viikon lopputarkistus: Peli päättyy kerran, kun aika loppuu. Nopea kaksoispainallus ei anna kahta tulosta, ja uusi peli nollaa ajan sekä pisteet.

Työnäyte Git-repositoryyn ennen rastia: pelisäännöt, ajastimen ja palautteen commitit sekä kolmen rajatestin tulokset.

Vko 48 · 23.–27.11. – Feature 4 · Kasvava kiire

Yhteys kokonaisprojektiin: Viikon 47 kello ja pisteet tekevät pelistä kierroksen, mutta kiire pysyy samana alusta loppuun. Toimeksianto vaatii kasvavaa vaikeutta, joten vertaat ensin kahta tapaa ja rakennat valitun tavan tasot yhteen Inspectorissa säädettävään listaan. Kun tasot ovat yhdessä paikassa, niitä voi säätää testien ja palautteen jälkeen koskematta koodiin.

Viikon tavoite: Peli vaikeutuu pisteiden mukaan kolmella tasolla: tilaukset isonevat tai asiakkaat odottavat lyhyemmän ajan.

Työvaihe 1 / 3 · Vertaa kahta tapaa vaikeuttaa peliä ja tee valinta.

- ☐ Lue kaksi tapaa vaikeuttaa peliä: A = tilauksissa on enemmän tuotteita, B = asiakkaalla on vähemmän aikaa.
- ☐ Pyydä vertainen eli toinen opiskelija tai ohjaaja keskustelemaan kanssasi kymmeneksi minuutiksi.
- ☐ Arvioikaa kumpikin tapa kolmella kysymyksellä: Huomaako pelaaja vaikeutumisen? Onko arvoja helppo säätää? Onko sitä helppo testata?
- ☐ Valitse A, B tai molemmat. Kirjoita päiväkirjaan valinta ja kaksi perustelua.
- ☐ Kirjoita päiväkirjaan keskustelukumppanin rooli, esimerkiksi vertainen tai ohjaaja.

Valmis kun: Päiväkirjassa ovat molemmat tavat, kolmen kysymyksen vastaukset, valinta ja kaksi perustelua.

Tallenna työnäyte: Vertailu ja valinta viikon 48 päiväkirjaan.

Työvaihe 2 / 3 · Tee kolme vaikeustasoa, joita voi säätää Inspectorissa.

- ☐ Luo skripti DifficultyController avun työpohjasta ja liitä se GameManager-objektiin.
- ☐ Lisää GameManageriin kenttä [SerializeField] private DifficultyController difficultyController; ja raahaa GameManager-objekti Inspectorissa sen Difficulty Controller -kenttään.
- ☐ Aseta Inspectorissa Levels-listan kooksi 3. Täytä tasot avun taulukon mukaan tai omien arvojesi mukaan.
- ☐ Täydennä GetLevel: palauta korkein taso, jonka aloituspisteet pelaaja on jo saavuttanut. Aloituspisteet ovat tason minScore-arvo.
- ☐ Jos valitsit tavan A: muuta CreateOrder muotoon CreateOrder(int count) ja poista sen Random.Range-arvonta. GameManager antaa sille arvon difficultyController.GetLevel(score).itemCount.
- ☐ Jos valitsit tavan B: aseta jokaiselle asiakkaalle odotusaika tason customerTime-arvosta. Kun aika loppuu, asiakas lähtee ja tulee uusi tilaus.
- ☐ Muuta yhden tason arvoa Inspectorissa ja paina Play. Muutos näkyy pelissä ilman koodimuutosta.

Valmis kun: Omilla raja-arvoillasi (esimerkiksi 0, 31 ja 61) peli käyttää eri tasoa, ja tason arvot voi muuttaa Inspectorissa.

Tallenna työnäyte: Commit ja push. Commit-linkki ja tasotaulukko viikon 48 päiväkirjaan.

Työvaihe 3 / 3 · Testaa, että pelin kolme tilaa pysyvät erillään.

- ☐ Kirjoita päiväkirjaan odotettu tulos ennen kuin testaat.
- ☐ Pelaa kolme kierrosta niin, että pääset eri kierroksilla tasoille 0, 1 ja 2 eli Levels-listan kolmelle riville minScore-arvojesi mukaan.
- ☐ Tarkista joka kierroksella: kun odotat valikossa 10 sekuntia ja painat Aloita, aika alkaa täydestä. Pelissä näkyy vain peliruutu, ja tulosruudulla pisteet eivät enää muutu.
- ☐ Aloita uusi peli tason 2 jälkeen. Tarkista, että peli alkaa taas tasolta 0.
- ☐ Kirjaa tulokset päiväkirjaan. Jos jokin ei mennyt odotetusti, korjaa, tee commit ja testaa uudelleen.
- ☐ Kirjoita GitHubissa jokaiseen tämän viikon valmiiseen issueen kommentti. Lisää kommenttiin linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Kolmen kierroksen tulokset on kirjattu, ja uusi peli alkaa aina tasolta 0.

Tallenna työnäyte: Testien tulokset ja commit-linkit viikon 48 päiväkirjaan.

Viikon lopputarkistus: Valinta on perusteltu. Omilla raja-arvoillasi (esimerkiksi 0, 31 ja 61) tilauksen koko tai asiakkaan odotusaika muuttuu tasojen mukaan joka kerta, eikä jokaiselle tasolle tarvita omaa if-lausetta.

Työnäyte Git-repositoryyn ennen rastia: vertailu ja valinta päiväkirjassa, vaikeustasojen commit ja testitulokset.

Vko 49 · 30.11.–4.12. – Feature 5 · Top 5 -tuloslista

Yhteys kokonaisprojektiin: Viikon 48 jälkeen kierros päättyy pisteisiin, mutta tulos katoaa, kun peli suljetaan. Nyt tulosruutu saa nimimerkin ja top 5 -listan, jonka SaveService tallentaa PlayerPrefsiin selaimen muistiin, koska toimeksianto vaatii parhaiden tulosten tallentamista. Tämän jälkeen toimeksiannon kaikki vaatimukset toimivat, ja asiakas pelaa koko peliä viikolla 50.

Viikon tavoite: Pelaaja tallentaa tuloksensa nimimerkillä, ja viisi parasta tulosta näkyy yhä, kun peli avataan uudelleen.

Työvaihe 1 / 5 · Lisää tulosruutuun nimimerkkikenttä ja Tallenna-painike.

- ☐ Lisää ResultPaneliin syöttökenttä: UI → Input Field - TextMeshPro. Nimeä se NicknameInput.
- ☐ Valitse NicknameInputin alta Text Area → Placeholder ja kirjoita Inspectorin tekstikenttään vihjetekstiksi Nimimerkki.
- ☐ Lisää ResultPaneliin painike SaveScoreButton tekstillä Tallenna tulos.
- ☐ Lisää ResultPaneliin teksti HighScoresText, johon tulee viisi parasta tulosta. Siirrä Scene-ikkunassa tulosruudun tekstit, kenttä ja painikkeet erilleen ja tee HighScoresText viiden rivin korkuiseksi.

Valmis kun: Tulosruudulla näkyvät nimimerkkikenttä, Tallenna tulos -painike ja tyhjä tuloslista.

Tallenna työnäyte: Kuvakaappaus tulosruudusta polkuun project-docs/evidence/week-49/tulosruutu.png, commit ja push. Polku viikon 49 päiväkirjaan.

Työvaihe 2 / 5 · Tee SaveService-skripti, joka tallentaa viisi parasta tulosta.

- ☐ Luo skripti SaveService avun työpohjasta ja liitä se uuteen tyhjään objektiin SaveService.
- ☐ Lue työpohjasta kaksi luokkaa: ScoreEntry on yksi tulos (nimimerkki ja pisteet) ja ScoreList on tulosten lista.
- ☐ Täydennä SaveScore: lisää uusi tulos listaan ja järjestä lista pisteiden mukaan suurimmasta pienimpään. Poista sitten kuudes tulos ja sitä huonommat.
- ☐ Etsi SaveScore-metodista valmiit tallennusrivit ja kirjoita päiväkirjaan omin sanoin, mitä JsonUtility.ToJson, PlayerPrefs.SetString ja PlayerPrefs.Save tekevät.
- ☐ Täydennä LoadScores: lue teksti PlayerPrefs.GetString-metodilla ja muuta se takaisin listaksi. Jos tallennusta ei vielä ole, palauta tyhjä lista.

Valmis kun: SaveScore ja LoadScores on täydennetty, ja päiväkirjassa on omin sanoin selitys tallennusriveistä.

Tallenna työnäyte: Commit ja push. Commit-linkki ja selitys viikon 49 päiväkirjaan.

Työvaihe 3 / 5 · Kytke tulosruutu tallentamaan ja näyttämään parhaat tulokset.

- ☐ Lisää GameManageriin avun työpohjan kentät nicknameInput, highScoresText ja saveService.
- ☐ Raahaa Inspectorissa NicknameInput, HighScoresText ja SaveService-objekti GameManager-skriptin kenttiin Nickname Input, High Scores Text ja Save Service.
- ☐ Lisää GameManageriin avun ShowHighScores-metodi. Se kirjoittaa listan rivit HighScoresText-tekstiin.
- ☐ Täydennä avun SaveCurrentScore-metodi: se tallentaa nimimerkin ja pisteet SaveServicellä ja näyttää palautetun listan.
- ☐ Kytke SaveScoreButtonin On Click () -listaan GameManager → SaveCurrentScore.
- ☐ Täydennä EndGame-metodia: näytä tallennettu lista heti, kun tulosruutu avautuu.
- ☐ Paina Play ja tallenna kuusi eri tulosta. Tarkista, että lista näyttää viisi parasta suurimmasta pienimpään.

Valmis kun: Tulosruutu näyttää tallennetut tulokset heti, ja kuudesta tuloksesta näkyy viisi parasta oikeassa järjestyksessä.

Tallenna työnäyte: Commit ja push. Kuvakaappaus listasta kansioon project-docs/evidence/week-49/ ja polku viikon 49 päiväkirjaan.

Työvaihe 4 / 5 · Tarkista nimimerkki ennen tallennusta.

- ☐ Päätä nimimerkin enimmäispituus, esimerkiksi 12 merkkiä. Kirjaa päätös päiväkirjaan.
- ☐ Lisää avun CleanNickname-metodi SaveService-skriptiin ja kutsu sitä SaveScore-metodin ensimmäisellä rivillä. Metodi poistaa jo valmiiksi välilyönnit alusta ja lopusta (Trim).
- ☐ Jos nimimerkki on tyhjä, käytä nimeä Nimetön.
- ☐ Jos nimimerkki on liian pitkä, lyhennä se sallittuun pituuteen.
- ☐ Aseta NicknameInput-kentän Character Limit -arvoksi sama enimmäispituus.
- ☐ Kirjaa päiväkirjaan, mitä peli tallentaa (nimimerkki ja pisteet) ja mitä se ei tallenna (oikea nimi, salasanat tai muut henkilötiedot).

Valmis kun: Tyhjä nimi tallentuu nimellä Nimetön, ja liian pitkä nimi lyhenee sallittuun pituuteen.

Tallenna työnäyte: Commit ja push. Päätökset ja commit-linkki viikon 49 päiväkirjaan.

Työvaihe 5 / 5 · Testaa tallennus selaimessa ja perustele ratkaisu.

- ☐ Kirjoita päiväkirjaan odotettu tulos kolmelle testille ennen kuin testaat.
- ☐ Testi 1: tallenna kuusi eri tulosta. Vain viisi parasta näkyy.
- ☐ Testi 2: tallenna kaksi yhtä suurta tulosta. Tarkista, että järjestys noudattaa asiakkaan kanssa sovittua sääntöä. Jos sääntöä ei ole sovittu, aiempi tulos on ylempänä, ja asia on avoin.
- ☐ Testi 3: kopioi pelin osoite, sulje välilehti ja avaa sama osoite uudelleen samassa selaimessa ilman uutta buildia. Sama lista näkyy.
- ☐ Kirjoita päiväkirjan Miksi tein näin? -kenttään, miksi PlayerPrefs sopii tähän peliin. Kirjoita myös, että tiedot säilyvät vain tässä selaimessa ja että pelaaja voi itse muuttaa tai poistaa ne.
- ☐ Kirjoita GitHubissa jokaiseen tämän viikon valmiiseen issueen kommentti. Lisää kommenttiin linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.
- ☐ Vertaa sovittua PlayerPrefsiä vähintään yhteen vaihtoehtoon, esimerkiksi JSON-tiedostoon tai palvelimeen, ja kirjaa päiväkirjaan, miksi se sopii pieneen selainpeliin. Jos vertailu puoltaa selvästi muuta, poikkeamasta sovitaan ohjaajan kanssa ennen seuraavaa toteutusviikkoa.

Valmis kun: Kolmen testin tulokset ja ratkaisun perustelu vertailuineen ovat päiväkirjassa.

Tallenna työnäyte: Testitulokset, perustelu, vertailu ja commit-linkit viikon 49 päiväkirjaan.

Viikon lopputarkistus: Kuudesta tuloksesta näkyy vain viisi parasta myös silloin, kun sivu avataan uudelleen. Tasapisteet ja liian pitkä nimimerkki on testattu, ja PlayerPrefs on perusteltu vertaamalla sitä vähintään yhteen vaihtoehtoon.

Työnäyte Git-repositoryyn ennen rastia: tallennuksen commitit, neljän testin tulokset (kuusi tulosta, tasapisteet, uudelleenavaus ja nimimerkin tarkistus) sekä ratkaisun perustelu ja vertailu päiväkirjassa.

Vaihe 3 – Asiakkaan palaute ja käytettävyys

Vko 50 · 7.–11.12. – Katselmointi · Asiakas pelaa peliäsi

Yhteys kokonaisprojektiin: Viikoilla 45–49 rakensit toimeksiannon kaikki vaatimukset, ja nyt asiakas kokeilee niitä ensimmäistä kertaa omin käsin. Tarkkailet neuvomatta, missä tilaus, tuotteiden valinta tai palaute jää epäselväksi, ja erotat asiakkaan palautteen omasta tulkinnastasi. Sovittu muutos tehdään heti seuraavalla viikolla 51, ennen joululomaa.

Viikon tavoite: Asiakas pelaa pelin alusta loppuun, ja yhdessä sovitaan yksi rajattu muutos GitHub-issueksi.

Työvaihe 1 / 3 · Valmistele asiakkaalle kokeiltava versio.

- ☐ Käy läpi tarkistuslista. Tarkista, että valikko, 1–3 tuotteen tilaus, toimitus, pisteet, ajastin, vaikeus, tulosruutu ja tallennus toimivat.
- ☐ Tee selainversio (Build And Run) ja pelaa se kerran itse alusta loppuun.
- ☐ Kirjoita muistiin version tunniste: päivämäärä ja viimeisimmän commitin tunniste.
- ☐ Sovi ohjaajan kanssa katselmoinnin aika ja se, kuka nimetty edustaja esittää asiakasta. Katselmointi on tapaaminen, jossa asiakas kokeilee keskeneräistä peliä ja antaa palautetta.
- ☐ Valmistele viiden minuutin esittely: mitä peli tekee, mikä on valmista ja mikä vielä puuttuu.

Valmis kun: Selainversio toimii alusta loppuun, ja katselmoinnin aika on sovittu.

Tallenna työnäyte: Version tunniste ja tarkistuslistan tulos viikon 50 päiväkirjaan.

Työvaihe 2 / 3 · Anna asiakkaan pelata ja kirjaa, mitä hän sanoo.

- ☐ Pyydä asiakasta pelaamaan peli alusta loppuun. Älä neuvo, ellei hän pyydä apua.
- ☐ Kirjoita ylös asiakkaan palaute ilman nimiä, esimerkiksi: asiakkaan edustaja ei huomannut tilausta. Sanatarkat lausumat lähetät tarvittaessa ohjaajalle Teamsissa.
- ☐ Kirjoita erikseen, mitä itse näit: missä asiakas epäröi tai painoi väärää painiketta.
- ☐ Selitä yksi tekninen ratkaisu arkikielellä, esimerkiksi miten tulokset tallentuvat. Kirjaa, ymmärsikö asiakas.
- ☐ Kysy lopuksi: mikä yksi asia pitäisi muuttaa ensin?

Valmis kun: Muistiinpanoissa ovat asiakkaan palaute ja omat havaintosi erikseen, ja henkilöistä on kirjattu vain roolit.

Tallenna työnäyte: Muistiinpanot viikon 50 päiväkirjaan.

Työvaihe 3 / 3 · Sovi asiakkaan kanssa yksi muutos ja tee siitä issue.

- ☐ Valitse palautteesta yhdessä asiakkaan kanssa yksi muutos, jonka ehdit tehdä viikolla 51.
- ☐ Tee muutoksesta GitHub-issue: otsikko verbillä, asiakkaan palaute ilman nimiä, arvio ja Valmis kun -ehto.
- ☐ Jos asiakas pitää muutosta välttämättömänä, se on pakollinen (P0): anna label P0 pakollinen. Muuten se on tärkeä (P1): anna label P1 tärkeä.
- ☐ Pyydä asiakkaalta hyväksyntä issuelle. Kirjaa hyväksyjän rooli ja päivä.
- ☐ Vertaa kolmen viikolla 44 tehdyn issuen arviota siihen, kauanko työ oikeasti kesti. Kirjaa ero ja sen syy päiväkirjaan.
- ☐ Kirjoita katselmointimuistio päiväkirjaan: päivä, osallistujien roolit, asiakkaan palaute, oma tulkinta ja päätös.

Valmis kun: GitHubissa on yksi hyväksytty muutosissue, ja katselmointimuistio on päiväkirjassa.

Tallenna työnäyte: Issuen linkki ja katselmointimuistio viikon 50 päiväkirjaan.

Viikon lopputarkistus: Asiakas on pelannut pelin alusta loppuun. Muistiossa näkyvät asiakkaan palaute, oma tulkinta, päätös, hyväksyjän rooli ja yksi rajattu issue.

Työnäyte Git-repositoryyn ennen rastia: version tunniste, katselmointimuistio ja hyväksytty muutosissue.

Vko 51 · 14.–18.12. – Feature 6 · Asiakkaan toivoma parannus

Yhteys kokonaisprojektiin: Viikon 50 katselmoinnissa asiakas valitsi yhden muutoksen, ja nyt toteutat sen omassa Git-haarassa. Haara pitää toimivan main-version turvassa, ja pull request näyttää ketjun palautteesta issueen, koodiin ja testiin. Kun muutos on yhdistetty, viikolla 2 uusi pelaaja kokeilee peliä ilman neuvoja.

Viikon tavoite: Asiakkaan pyytämä muutos on pelattavana, ja vanha pelipolku toimii edelleen.

Työvaihe 1 / 3 · Tarkista muutosissue ja kirjoita sille testi.

- ☐ Avaa viikolla 50 tekemäsi muutosissue GitHubissa ja lue siihen kirjattu asiakkaan palaute.
- ☐ Arvioi, onko muutos enintään yhden päivän työ. Jos se on isompi, jaa se kahdeksi issueksi.
- ☐ Kirjoita issueen hyväksymistesti: mitä asiakas tekee ja mitä hän näkee, kun muutos on valmis.
- ☐ Kirjoita issueen myös vanhan pelipolun testi: Aloita → toimita tilaus → tulosruutu toimii kuten ennen.

Valmis kun: Issuessa ovat arvio, hyväksymistesti ja vanhan pelipolun testi.

Tallenna työnäyte: Issuen linkki viikon 51 päiväkirjaan.

Työvaihe 2 / 3 · Tee muutos omassa Git-haarassa.

- ☐ Varmista ensin, että main-haara eli pääversio toimii: pelaa yksi kierros.
- ☐ Luo uusi haara eli branch: valitse GitHub Desktopissa Current Branch → New Branch. Anna nimeksi esimerkiksi feature/suurempi-tilaus.
- ☐ Tee muutos pienissä osissa. Tee commit aina, kun yksi osa toimii.
- ☐ Testaa jokaisen commitin jälkeen, että tilaus, toimitus ja pisteet toimivat edelleen.
- ☐ Tee push, jolloin haara näkyy GitHubissa. Uuden haaran ensimmäisessä pushissa GitHub Desktopin painike on nimeltään Publish branch.

Valmis kun: Haara näkyy GitHubissa, ja siinä on vähintään kaksi pientä committia.

Tallenna työnäyte: Haaran nimi ja commit-linkit viikon 51 päiväkirjaan.

Työvaihe 3 / 3 · Pyydä katselmointi pull requestilla ja yhdistä muutos.

- ☐ Avaa repository GitHubissa ja paina Compare & pull request. Jos painiketta ei näy, valitse Pull requests → New pull request ja vertailtavaksi oma haarasi. Pull request on pyyntö yhdistää oma haara main-haaraan.
- ☐ Kirjoita pull requestiin, mitä muutit ja miten testasit. Lisää rivi Closes #numero, jossa numero on muutosissuen numero. Se sulkee issuen, kun pull request yhdistetään.
- ☐ Pyydä ohjaajaa tai vertaista kommentoimaan. Vastaa jokaiseen kommenttiin tai korjaa koodi samaan haaraan ja tee push: korjaus näkyy pull requestissa itsestään.
- ☐ Aja hyväksymistesti ja vanhan pelipolun testi vielä kerran.
- ☐ Paina Merge pull request ja Confirm merge, jolloin muutos siirtyy main-haaraan. Vaihda sitten GitHub Desktopissa haaraksi main ja paina Fetch origin ja Pull origin, jotta koneesi main on ajan tasalla.
- ☐ Kirjaa päiväkirjaan ketju: palaute → issue → haara → pull request → commit → testi. Lisää jokaisesta linkki.

Valmis kun: Pull request on yhdistetty, siinä on kommentti ja vastaus, ja molemmat testit menivät odotetusti.

Tallenna työnäyte: Pull requestin linkki ja koko ketju viikon 51 päiväkirjaan.

Viikon lopputarkistus: Muutos täyttää Valmis kun -ehdon, vanha pelipolku toimii, katselmointikommenttiin on vastattu ja muutos on yhdistetty main-haaraan.

Työnäyte Git-repositoryyn ennen rastia: issue, haara, pull request kommentteineen, commitit ja testitulokset.

Vko 52 · 21.–25.12. – Joululoma

Ei projektityötä eikä korvaavia tehtäviä. Jatka viikolla 2 viimeisimmästä toimivasta main-versiosta.

Vko 53 · 28.12.–1.1. – Joululoma

Ei projektityötä eikä korvaavia tehtäviä. Jatka viikolla 2 viimeisimmästä toimivasta main-versiosta.

Vko 1 · 4.–8.1. – Joululoma

Ei projektityötä eikä korvaavia tehtäviä. Jatka viikolla 2 viimeisimmästä toimivasta main-versiosta.

Vko 2 · 11.–15.1. – Feature 7 · Peli ohjaa pelaajaa itse

Yhteys kokonaisprojektiin: Asiakkaan muutos on nyt pelissä, mutta et itse enää huomaa, mikä pelissä on epäselvää. Vertaat peliä viikon 44 luonnokseen ja katsot, miten uusi pelaaja selviää ilman neuvoja, koska julkaistu peli ei saa vaatia tekijää vieressä. Kaksi korjausta ja uusintatesti vievät pelin viikon 3 järjestelmälliseen testaukseen.

Viikon tavoite: Uusi pelaaja ymmärtää tavoitteen ja pelaa kierroksen ilman, että kukaan neuvoo vieressä.

Kenney.nl – käyttöliittymäpaketit ja ikonit (CC0)

Game-icons.net – tuhansia ikoneita (CC BY, mainitse tekijä)

Työvaihe 1 / 3 · Vertaa peliä viikon 44 luonnokseen.

- ☐ Avaa viikon 44 luonnos project-docs/evidence/week-44/mockup.png.
- ☐ Ota kuvakaappaus pelisi valikosta, peliruudusta ja tulosruudusta.
- ☐ Kirjoita jokaisesta ruudusta, mikä on erilaista kuin luonnoksessa.
- ☐ Kirjoita jokaisen eron perään, miksi muutit sen, tai merkitse se korjattavaksi.

Valmis kun: Jokaisesta kolmesta ruudusta on kirjattu erot ja niiden syyt.

Tallenna työnäyte: Kuvakaappaukset kansioon project-docs/evidence/week-2/, commit ja push. Vertailu viikon 2 päiväkirjaan.

Työvaihe 2 / 3 · Testaa peliä uudella pelaajalla ilman neuvoja.

- ☐ Pyydä kokeilijaksi vertainen, joka ei ole pelannut peliäsi.
- ☐ Anna hänelle yksi tehtävä: ”Aloita peli, toimita yksi tilaus ja katso tuloksesi.” Älä neuvo.
- ☐ Kirjaa havainnot: missä hän epäröi, mitä hän painoi väärin ja mitä hän kysyi.
- ☐ Mittaa, kauanko aloituksesta kestää ensimmäiseen oikeaan toimitukseen.

Valmis kun: Päiväkirjassa ovat testaajan rooli ja päivä, annettu tehtävä, havainnot ja aika ensimmäiseen toimitukseen.

Tallenna työnäyte: Havainnot viikon 2 päiväkirjaan.

Työvaihe 3 / 3 · Korjaa kaksi suurinta ongelmaa ja testaa uudelleen.

- ☐ Valitse havainnoista kaksi, jotka haittasivat pelaamista eniten.
- ☐ Ota kuvakaappaus molemmista kohdista ennen muutosta.
- ☐ Tee korjaukset, esimerkiksi isompi teksti, selkeämpi painike tai lyhyt ohjeteksti.
- ☐ Ota kuvakaappaus molemmista kohdista muutoksen jälkeen. Tee commit ja push.
- ☐ Anna sama tehtävä toiselle uudelle pelaajalle ja kirjaa, auttoivatko korjaukset.
- ☐ Kirjoita GitHubissa jokaiseen tämän viikon valmiiseen issueen kommentti. Lisää kommenttiin linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Kahdesta korjauksesta on kuvat ennen korjausta ja sen jälkeen, ja uusintatestin tulos on kirjattu.

Tallenna työnäyte: Ennen/jälkeen-kuvat kansioon project-docs/evidence/week-2/. Uusintatestin tulos ja commit-linkki viikon 2 päiväkirjaan.

Viikon lopputarkistus: Uusi pelaaja ymmärtää tavoitteen ja pelaa yhden tilauksen loppuun ilman neuvoja. Kahdesta korjauksesta on kuvat ennen korjausta ja sen jälkeen.

Työnäyte Git-repositoryyn ennen rastia: vertailu luonnokseen, havainnot, ennen/jälkeen-kuvat ja uusintatestin tulos.

Vaihe 4 – Testaus ja koodin laatu

Vko 3 · 18.–22.1. – Pelin laatu · Peli kestää pelaamista

Yhteys kokonaisprojektiin: Tähän asti olet testannut kunkin viikon uuden toiminnon erikseen. Nyt kirjoitat 12 testitapausta koko pelille ennen ajoa ja ajat ne selainversiolla, koska järjestelmällinen testaus näyttää, kestääkö peli rajatilanteet ja rikkinäiset tiedostot. Testaustaulukon tapauksia käytät viikon 4 refaktoroinnissa ja julkaisun korjauksissa.

Viikon tavoite: Pelin koko kierros on testattu 12 testitapauksella, ja kolme virhettä on korjattu ketjuna uusintatesteineen.

Avaa näyttöaineisto

Työvaihe 1 / 3 · Kirjoita 12 testitapausta ennen testaamista.

- ☐ Luo tiedosto project-docs/testaus.md avun taulukkopohjasta.
- ☐ Numeroi testitapaukset T01, T02, T03 ja niin edelleen. T tarkoittaa testitapausta ja numero sen järjestystä. Tunnuksella viittaat testiin myöhemmin.
- ☐ Kirjoita neljä testitapausta tavalliselle pelaamiselle: aloitus, oikea toimitus, väärä toimitus ja uusi peli.
- ☐ Kirjoita neljä testitapausta rajatilanteille: aika 0, kaksoispainallus, tyhjä nimimerkki ja liian pitkä nimimerkki.
- ☐ Kirjoita neljä testitapausta datalle eli tiedostoille ja tallennukselle: puuttuva products.json, rikkinäinen products.json, tyhjä tallennus ja kuudes tulos top 5 -listaan.
- ☐ Täytä jokaiseen testitapaukseen sarakkeet Lähtötila, Mitä teen ja Odotettu tulos ennen kuin ajat yhtään testiä. Käytä omia arvojasi: kierroksen pituus, väärän toimituksen pisteet ja nimimerkin enimmäispituus.

Valmis kun: Tiedostossa on 12 testitapausta T01–T12, ja jokaisella on odotettu tulos.

Tallenna työnäyte: project-docs/testaus.md, commit ja push.

Työvaihe 2 / 3 · Aja testitapaukset selainversiolla ja kirjaa tulokset.

- ☐ Tee selainversio (Build And Run) ja kirjaa version tunniste testaustaulukon alkuun.
- ☐ Aloita puhtaasta tilanteesta: avaa peli yksityisessä selainikkunassa, jolloin vanhat tulokset eivät ole mukana.
- ☐ Aja testitapaukset T01–T12 järjestyksessä. Kirjaa jokaisesta, mitä tapahtui ja läpäisikö testi.
- ☐ Tyhjennä datatestejä varten Products Json -kenttä tai poista products.json-tiedostosta pilkku. Tee uusi selainversio ja palauta kenttä tai tiedosto testin jälkeen.
- ☐ Älä merkitse testiä läpäistyksi, jos et ajanut sitä itse.

Valmis kun: Jokaisella testitapauksella T01–T12 on todellinen tulos ja merkintä, läpäisikö se.

Tallenna työnäyte: Päivitetty project-docs/testaus.md, commit ja push.

Työvaihe 3 / 3 · Korjaa kolme virhettä ja kirjaa jokaisesta koko ketju.

- ☐ Valitse kolme testitapausta, jotka eivät läpäisseet. Jos aitoja virheitä ei ole kolmea, pyydä ohjaajalta vikatehtävä eli ohjaajan valmistelema virhe, jonka etsit ja korjaat.
- ☐ Kirjaa jokaisesta, miten virhe toistetaan, mikä oli odotettu tulos ja mitä tapahtui.
- ☐ Etsi syy ennen kuin korjaat. Kirjoita syy yhdellä virkkeellä.
- ☐ Korjaa ja tee commit, jonka viestissä on testitapauksen tunnus. Esimerkiksi: ”Korjaa testitapaus T05, tulosruutu aukeaa vain kerran.”
- ☐ Aja sama testitapaus uudelleen. Aja myös regressiotesti eli toinen testitapaus, joka käyttää samaa koodia: se näyttää, ettei korjaus rikkonut muuta.
- ☐ Kirjaa jokainen ketju päiväkirjaan: havainto → toistamisohje → syy → korjauscommit → uusintatesti → regressiotesti.

Valmis kun: Päiväkirjassa on kolme täydellistä ketjua, ja jokaisen uusintatesti meni odotetusti.

Tallenna työnäyte: Kolme ketjua commit-linkkeineen viikon 3 päiväkirjaan.

Viikon lopputarkistus: Kaikissa 12 testitapauksessa näkyvät lähtötila, toiminta, odotettu tulos, todellinen tulos ja läpäisy. Kolmessa ketjussa näkyvät havainto, syy, korjauscommit ja onnistunut uusintatesti.

Työnäyte Git-repositoryyn ennen rastia: project-docs/testaus.md, kolme virheenkorjausketjua ja korjauscommitit.

Vko 4 · 25.–29.1. – Pelin laatu · Koodi, jonka toinen ymmärtää

Yhteys kokonaisprojektiin: Viikon 3 testaustaulukko antaa testitapauksen, jolla näet, toimiiko peli refaktoroinnin jälkeen kuten ennen. Selkeytät yhden koodikohdan, esimerkiksi pitkän metodin tai epäselvän nimen, ja pyydät toisen ihmisen katselmoimaan sen, koska selkeä koodi helpottaa julkaisua edeltäviä korjauksia. Samalla harjoittelet ratkaisun selittämistä, jota tarvitset näytössä viikolla 7.

Viikon tavoite: Yksi vaikeasti luettava koodikohta on selkeytetty, peli toimii kuten ennen, ja osaat selittää ratkaisusi.

Työvaihe 1 / 3 · Etsi koodista yksi kohta, jota on vaikea ymmärtää.

- ☐ Avaa C#-skriptit ja etsi yksi näistä: sama koodi kahdessa paikassa, nimi, joka ei kerro tehtäväänsä (esimerkiksi a tai temp) tai yli 30 rivin metodi.
- ☐ Etsi myös skripti, joka tekee monta eri asiaa, esimerkiksi laskee pisteet ja päivittää tekstit.
- ☐ Valitse näistä yksi kohta. Kirjoita päiväkirjaan, mikä kohdassa on vaikeaa ja miksi.
- ☐ Valitse testaustaulukosta testitapaus, joka kulkee tämän koodin kautta. Kirjaa testitapauksen tunnus, esimerkiksi T03.

Valmis kun: Päiväkirjassa on yksi valittu kohta, sen ongelma ja siihen liittyvä testitapaus.

Tallenna työnäyte: Kuvaus ja testitapauksen tunnus viikon 4 päiväkirjaan.

Työvaihe 2 / 3 · Selkeytä kohta niin, että peli toimii kuten ennen.

- ☐ Aja valitsemasi testitapaus ennen muutosta ja kirjaa tulos.
- ☐ Tee yksi rajattu muutos, esimerkiksi anna metodille kuvaava nimi tai siirrä toistuva koodi omaan metodiinsa.
- ☐ Aja sama testitapaus muutoksen jälkeen. Tuloksen pitää olla sama kuin ennen.
- ☐ Tee commit, jonka viesti kertoo muutoksen, esimerkiksi ”Selkeytä pisteiden laskua: CalculateOrderScore”.

Valmis kun: Sama testitapaus läpäisee ennen ja jälkeen muutoksen, ja muutos on yhdessä commitissa.

Tallenna työnäyte: Commit-linkki sekä ennen- ja jälkeen-tulokset viikon 4 päiväkirjaan.

Työvaihe 3 / 3 · Pyydä koodikatselmointi ja selitä ratkaisusi.

- ☐ Pyydä ohjaajaa tai vertaista lukemaan muuttamasi koodi. Koodikatselmointi tarkoittaa, että toinen ihminen lukee koodin ja kommentoi sitä.
- ☐ Näytä hänelle koodi ennen ja jälkeen muutoksen, esimerkiksi GitHubin commit-näkymästä. Kirjaa saamasi kommentin sisältö ja katselmoijan rooli.
- ☐ Vastaa kommenttiin: korjaa koodia tai perustele, miksi pidät ratkaisun.
- ☐ Selitä samalla tapaamisella suullisesti yksi kohta, jossa käytit tekoälyä apuna. Jos et käyttänyt tekoälyä, selitä avun työpohjasta otettu kohta.
- ☐ Tarkista, että tekoälyn käyttö on kirjattu AI-lokiin eli sivuston AI-loki-näkymään, johon kirjaat tekoälyn avun.

Valmis kun: Päiväkirjassa ovat katselmoijan rooli, hänen kommenttinsa, oma vastauksesi ja se, minkä kohdan selitit suullisesti.

Tallenna työnäyte: Kommentti, vastaus ja selitetty kohta viikon 4 päiväkirjaan. Tekoälyn käyttö AI-lokiin.

Viikon lopputarkistus: Sama testitapaus läpäisee ennen ja jälkeen refaktoroinnin. Katselmointikommenttiin on vastattu, ja osaat selittää ratkaisusi ilman tekoälyä.

Työnäyte Git-repositoryyn ennen rastia: refaktoroinnin commit, testi ennen ja jälkeen, katselmoinnin kommentti ja vastaus.

Vaihe 5 – Julkaisu ja näyttö

Vko 5 · 1.–5.2. – Pelin julkaisu · Ensimmäinen julkaisuehdokas RC1 testipelattavana

Yhteys kokonaisprojektiin: Peli on nyt testattu ja selkeytetty, joten jäädytät ominaisuudet: peliin ei enää lisätä uutta, vaan vain virheitä korjataan. Merkitset ensimmäisen julkaisuehdokkaan (RC1) tagilla, ja asiakas ja toinen ihminen testaavat sen, koska kaksi testaajaa löytää eri virheitä. Luokittelemasi havainnot päättävät, mitä viikolla 6 korjataan ennen julkaisua.

Viikon tavoite: Julkaisuehdokas RC1 on merkitty tagilla, ja asiakas ja toinen testaaja ovat pelanneet sen.

Työvaihe 1 / 3 · Merkitse julkaisuehdokas RC1 ja jäädytä uudet ominaisuudet.

- □ Kirjaa päiväkirjaan päivä, josta alkaen peliin ei lisätä uusia ominaisuuksia vaan korjataan vain virheitä. Tätä kutsutaan ominaisuusjäädytykseksi.
- □ Pelaa main-haaran versio läpi ja varmista, että valikko, peliruutu, tulosruutu ja tallennus toimivat.
- □ Tarkista, että viimeisin commit on pushattu GitHubiin: merkitset sen seuraavassa osassa tagilla RC1. RC1 tarkoittaa ensimmäistä julkaisuehdokasta (release candidate 1): versiota, joka julkaistaan, jos testeissä ei löydy vakavia virheitä.
- □ Avaa GitHubissa Releases → Draft a new release. Kirjoita Choose a tag -kenttään RC1 ja valitse Create new tag. Rastita Set as a pre-release ja paina Publish release.
- □ Tee julkaisuehdokkaasta (RC1) selainversio Build And Run -toiminnolla. Testaajat pelaavat sitä sinun koneellasi.

Valmis kun: Tagi RC1 näkyy GitHubissa, ja se osoittaa main-haaran viimeisimpään committiin.

Tallenna työnäyte: RC1-tagin linkki ja commitin tunniste viikon 5 päiväkirjaan.

Työvaihe 2 / 3 · Anna asiakkaan ja yhden muun ihmisen testata julkaisuehdokas.

- □ Pyydä asiakkaan nimettyä edustajaa ja yhtä muuta ihmistä testaamaan julkaisuehdokas (RC1).
- □ Anna molemmille sama tehtävä: ”Pelaa peli alusta loppuun ja tallenna tulos.”
- □ Pyydä heitä sulkemaan peli ja avaamaan se uudelleen, jotta näet, toimiiko tallennus.
- □ Kirjaa jokainen havainto erikseen: kuka testasi (rooli, esimerkiksi asiakkaan edustaja tai testaaja A), mitä tapahtui ja missä kohdassa.

Valmis kun: Kahden testaajan havainnot on kirjattu erikseen, ja testaajien roolit näkyvät.

Tallenna työnäyte: Havainnot viikon 5 päiväkirjaan.

Työvaihe 3 / 3 · Päätä jokaisesta havainnosta, korjataanko se ennen julkaisua.

- □ Kirjoita jokainen havainto omalle rivilleen.
- □ Merkitse vakavuus: vakava (peli ei toimi tai kaatuu), haitallinen (peli toimii mutta hankalasti) tai pieni (ulkonäköasia).
- □ Merkitse toistuvuus: toistuu aina, joskus tai kerran.
- □ Päätä jokaisesta: korjataan nyt, tunnettu puute (kerrotaan käyttöohjeessa) tai myöhemmin (parannus, joka ei haittaa pelaajaa).
- □ Tee jokaisesta korjattavasta havainnosta GitHub-issue. Lisää sille uusi testitapaus project-docs/testaus.md-tiedostoon, esimerkiksi testitapaus T13.

Valmis kun: Jokaisella havainnolla on vakavuus, toistuvuus ja päätös, ja korjattavista on issuet.

Tallenna työnäyte: Luokiteltu lista ja issueiden linkit viikon 5 päiväkirjaan.

Viikon lopputarkistus: Ensimmäinen julkaisuehdokas (RC1) on merkitty yhteen committiin. Asiakas ja toinen käyttäjä ovat testanneet sen, ja jokaisella havainnolla on vakavuus, toistuvuus ja päätös.

Työnäyte Git-repositoryyn ennen rastia: RC1-tagi, kahden testaajan havainnot ja luokiteltu korjauslista.

Vko 6 · 8.–12.2. – Pelin julkaisu · v1.0 julki

Yhteys kokonaisprojektiin: Viikon 5 luokitellusta listasta korjaat vain korjataan nyt -havainnot. Sitten peli siirtyy Unity-editorista GitHub Pagesiin, ja testaat julkaistua linkkiä toisella laitteella, koska vain julkinen linkki osoittaa, että tiedostot, asetukset ja tallennus toimivat asiakkaan ympäristössä. Julkaistu versio v1.0 on sama, jonka esittelet näytössä viikolla 7.

Viikon tavoite: Peli on julkaistu GitHub Pagesissa versiona v1.0, ja toinen ihminen pelaa sen toisella laitteella käyttöohjeen avulla.

Työvaihe 1 / 4 · Korjaa vain ne virheet, jotka päätit korjata ennen julkaisua.

- ☐ Avaa viikon 5 luokiteltu lista. Ota työn alle vain havainnot, joiden päätös on ”korjataan nyt”.
- ☐ Korjaa yksi virhe kerrallaan ja tee jokaisesta oma commit.
- ☐ Aja jokaisen korjauksen jälkeen siihen liittyvä testitapaus ja regressiotesti eli toinen testitapaus, joka käyttää samaa koodia.
- ☐ Sulje korjatut GitHub-issuet ja lisää niihin linkki korjauscommitiin.

Valmis kun: Kaikki korjataan nyt -havainnot on korjattu ja testattu, ja niiden issuet on suljettu.

Tallenna työnäyte: Korjauscommitien linkit viikon 6 päiväkirjaan.

Työvaihe 2 / 4 · Julkaise peli GitHub Pagesiin ja merkitse se versioksi v1.0.

- ☐ Avaa File → Build Profiles, valitse Web ja tarkista, että CafeGame on scene-listassa.
- ☐ Avaa Player Settings → Web → Publishing Settings ja ota käyttöön Decompression Fallback. Ilman sitä GitHub Pages ei osaa avata pakattua peliä.
- ☐ Paina Build ja valitse kansioksi repositoryn docs-kansio. Luo VS Codella docs-kansioon tyhjä tiedosto .nojekyll: se kertoo GitHub Pagesille, että kansio julkaistaan sellaisenaan.
- ☐ Tee commit ja push. Tarkista GitHubista, että docs-kansiossa näkyvät index.html, Build ja TemplateData.
- ☐ Avaa GitHubissa Settings → Pages. Valitse Deploy from a branch, haaraksi main ja kansioksi /docs. Paina Save.
- ☐ Odota muutama minuutti, avaa Pages-sivun antama linkki ja pelaa yksi kierros.
- ☐ Kun peli aukeaa linkistä, merkitse sama commit tagilla v1.0: Releases → Draft a new release → tagi v1.0 → Publish release.

Valmis kun: Peli aukeaa julkaisulinkistä, ja tagi v1.0 osoittaa samaan committiin kuin julkaisu.

Tallenna työnäyte: Julkaisulinkki ja v1.0-tagin linkki viikon 6 päiväkirjaan.

Työvaihe 3 / 4 · Kirjoita README:hen käyttöohje ja tunnetut puutteet.

- ☐ Lisää README.md-tiedostoon otsikko ”Näin pelaat”. Lisää sen alle julkaisulinkki.
- ☐ Kirjoita ohje neljänä vaiheena: avaa linkki → Aloita → valitse tilauksen tuotteet → Toimita.
- ☐ Lisää otsikko ”Tunnetut puutteet” ja listaa sen alle viikon 5 tunnetut puutteet.
- ☐ Lisää otsikko ”Lähteet ja lisenssit”. Listaa sen alle käytetyt grafiikkapaketit ja niiden lisenssit.
- ☐ Pyydä toista ihmistä aloittamaan peli pelkän ohjeen avulla, ilman suullista apua. Kirjaa hänen roolinsa ja kohdat, joissa hän epäröi, ja korjaa ne ohjeeseen.
- ☐ Perusta tiedosto project-docs/nayttomatriisi.md: jokaista vaatimusta kohti rivi, jossa ovat tunnus, työnäytteen linkki ja viikko. Täytä jo rivit, joiden linkin tiedät, esimerkiksi julkaisu ja v1.0-tagi.

Valmis kun: README:ssä ovat julkaisulinkki, käyttöohje, tunnetut puutteet ja lähteet, ja project-docs/nayttomatriisi.md on perustettu.

Tallenna työnäyte: README.md ja project-docs/nayttomatriisi.md, commit ja push.

Työvaihe 4 / 4 · Testaa julkaistu peli toisella laitteella käyttöohjeen avulla.

- ☐ Pyydä toista ihmistä avaamaan julkaisulinkki eri laitteella tai eri selaimella.
- ☐ Pyydä häntä pelaamaan kierros README:n käyttöohjeen avulla aloituksesta tulosruutuun ja tallentamaan tulos.
- ☐ Pyydä häntä päivittämään sivu ja tarkistamaan, että tulos näkyy yhä.
- ☐ Kirjaa testattu laite ja selain, testaajan rooli ja tulos.

Valmis kun: Toinen ihminen on pelannut julkaistun pelin toisella laitteella tai selaimella, ja tulos on kirjattu.

Tallenna työnäyte: Laite, selain, testaajan rooli ja tulos viikon 6 päiväkirjaan.

Viikon lopputarkistus: v1.0-tagin commit vastaa julkaistua versiota. Toinen ihminen avaa linkin toisella selaimella tai laitteella ja pelaa kierroksen käyttöohjeen avulla.

Työnäyte Git-repositoryyn ennen rastia: julkaisulinkki, v1.0-tagi, testitulos toiselta laitteelta, README käyttöohjeineen ja perustettu project-docs/nayttomatriisi.md.

Vko 7 · 15.–19.2. – Näyttö · Aineisto ja luovutus

Yhteys kokonaisprojektiin: Peliä ei enää muuteta: julkaistu v1.0 ja viikkojen 43–51 ja 2–6 työnäytteet ovat näyttösi aineisto. Tarkistat päiväkirjan, kirjoitat itsearvioinnin ja liität jokaiseen näyttömatriisin vaatimukseen tarkan linkin, koska arvioija voi arvioida vain löydettävän osaamisen. Demossa näytät saman version ja selität ratkaisusi itse.

Viikon tavoite: Peli, repository ja projektipäiväkirja on luovutettu, ja jokaisella näyttömatriisin vaatimuksella on toimiva linkki.

Avaa näyttömatriisi

Avaa ja lataa AI-loki

Työvaihe 1 / 5 · Tarkista projektipäiväkirja ja vie se repositoryyn.

- ☐ Avaa Projektipäiväkirja-näkymä ja tarkista, että jokainen viikko on merkitty kirjatuksi.
- ☐ Täydennä puuttuvat kentät viikkonäkymissä.
- ☐ Tarkista jokaisen viikon Missä työnäyte on? -kentästä, että linkki aukeaa.

Valmis kun: Kaikki 15 viikkoa on kirjattu, ja jokaisen viikon työnäytelinkki aukeaa.

Tallenna työnäyte: Täydennetyt viikkomerkinnät sivuston projektipäiväkirjassa. Lataat tiedoston repositoryyn työvaiheessa 5.

Työvaihe 2 / 5 · Kirjoita itsearviointi omiin työnäytteisiin nojaten.

- ☐ Valitse projektista kolme asiaa, jotka osaat nyt hyvin.
- ☐ Liitä jokaiseen vahvuuteen työnäyte: linkki commitiin, testitapaukseen tai päiväkirjan viikkoon.
- ☐ Kirjoita yksi asia, jota haluat kehittää seuraavaksi. Kirjoita myös, miten aiot kehittää sitä.
- ☐ Kirjoita itsearviointi itse. Älä käytä tekoälyä tekstin kirjoittamiseen.

Valmis kun: Viikon 7 päiväkirjassa on kolme vahvuutta työnäytteineen ja yksi kehitysaskel.

Tallenna työnäyte: Itsearviointi viikon 7 päiväkirjaan.

Työvaihe 3 / 5 · Liitä jokaiseen näyttömatriisin kohtaan tarkka linkki.

- ☐ Avaa Näyttömatriisi-näkymä eli luettelo osaamisvaatimuksista, joihin tarvitset työnäytteen, ja lue ensimmäinen vaatimus.
- ☐ Etsi työnäyte, joka osoittaa, että vaatimus täyttyy: issue, C#-tiedosto, commit, testitapaus tai päiväkirjan viikko.
- ☐ Kopioi työnäytteen tarkka linkki. Linkin pitää avata juuri se kohta, ei repositoryn etusivua.
- ☐ Kirjoita tiedostoon project-docs/nayttomatriisi.md vaatimuksen riville tunnus, linkki ja viikko. Rastita sitten vaatimus Näyttömatriisi-näkymässä: rasti on oma muistilistasi.
- ☐ Toista osatehtävät 1–4 jokaiselle vaatimukselle.

Valmis kun: Tiedostossa project-docs/nayttomatriisi.md on jokaiselle 32 vaatimukselle rivi, ja jokaisen rivin tarkka linkki aukeaa.

Tallenna työnäyte: project-docs/nayttomatriisi.md, commit ja push. Linkki tiedostoon viikon 7 päiväkirjaan.

Työvaihe 4 / 5 · Harjoittele 8–10 minuutin demo.

- ☐ Kirjoita demon runko kuutena kohtana: pelin kulku, tuotelista JSON-tiedostossa, tallennus, yksi virheenkorjaus, Git-historia ja AI-loki.
- ☐ Harjoittele demo kerran ääneen toiselle ihmiselle ja ota aika. Kirjaa kuulijan rooli ja yksi hänen palautteensa. Tavoite on 8–10 minuuttia.
- ☐ Harjoittele, miten selität yhden C#-metodin omin sanoin.
- ☐ Varmista, että demossa käytät julkaistua versiota v1.0.

Valmis kun: Demo kestää 8–10 minuuttia ja käy läpi kaikki kuusi kohtaa.

Tallenna työnäyte: Demon runko viikon 7 päiväkirjaan.

Työvaihe 5 / 5 · Luovuta peli ja aineisto viimeistään pe 19.2.2027.

- ☐ Paina Projektipäiväkirja-näkymän painiketta Lataa koko päiväkirja (.md) ja korvaa repositoryn tiedosto project-docs/projektipaivakirja.md. Tee commit ja push. Tarkista, että repositoryssa ovat peli, README.md, gdd.md, testaus.md, projektipaivakirja.md ja nayttomatriisi.md.
- ☐ Avaa ladattu projektipaivakirja.md ja tarkista, että sen lopussa on otsikko AI-loki ja omat merkintäsi.
- ☐ Pyydä toista ihmistä avaamaan julkaisulinkki ja repository. Kirjaa, löysikö hän kaiken.
- ☐ Luovuta peli, repository, projektipäiväkirja ja näyttöaineisto ohjaajalle viimeistään pe 19.2.2027.

Valmis kun: Aineisto on luovutettu viimeistään pe 19.2.2027, ja toinen ihminen on tarkistanut sen.

Tallenna työnäyte: Luovutuksen päivä viikon 7 päiväkirjaan.

Viikon lopputarkistus: Tiedoston project-docs/nayttomatriisi.md jokaisella rivillä on tarkka linkki, joka aukeaa. Projektipäiväkirja ja AI-loki ovat repositoryssa, ja demo käyttää samaa v1.0-versiota.

Työnäyte Git-repositoryyn ennen rastia: peli, repository, project-docs/projektipaivakirja.md, project-docs/nayttomatriisi.md, AI-loki ja demo.

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

repository vko 43	*Projektin koodivarasto Gitissä.* Repository on projektin tiedostojen ja koko muutoshistorian säilytyspaikka. Tässä projektissa se on GitHubissa, ja sieltä löytyvät peliprojekti, project-docs-kansio ja työnäytteet.
commit vko 43	*Versionhallintaan tallennettu muutos.* Commit on yksi versionhallintaan tallennettu muutoskokonaisuus: mitä muutit ja miksi. Jokaisella commitilla on oma tunniste, johon voit linkittää työnäytteenä.
build vko 43	*Pelin ajettava julkaisuversio.* Build on Unityn tekemä valmis versio pelistä. Sitä pelataan ilman Unity-editoria, ja juuri sitä asiakas kokeilee.
WebGL vko 43	*Unityn selainjulkaisu.* WebGL on Unityn julkaisumuoto, joka toimii selaimessa ilman asennusta. WebGL-build on pelin selainversio – tämän projektin lopputuote.
push vko 43	*Commitien siirto GitHubiin.* Push siirtää koneellasi tehdyt commitit GitHubiin. Vasta pushin jälkeen ohjaaja ja asiakas näkevät muutoksesi.
P0 vko 43	*Pakollinen ydin.* P0 on se osa peliä, jonka on pakko valmistua: ilman sitä peliä ei voi luovuttaa asiakkaalle. Tee koko P0 valmiiksi ennen kuin aloitat lisäominaisuuksia.
P1 vko 44	*Tärkeä jatkosisältö.* P1 on ominaisuus, joka tehdään vasta, kun koko P0 toimii. Se parantaa peliä, mutta peli on luovutettavissa myös ilman sitä.
P2 vko 44	*Valinnainen lisä.* P2 on ominaisuus, joka voidaan jättää kokonaan pois, jos aika loppuu. Merkitse P2:ksi kaikki, mistä voi luopua ilman, että asiakkaan vaatimus jää täyttämättä.
GDD vko 44	*Game Design Document, pelin suunnitteludokumentti.* GDD kokoaa yhteen tiedostoon pelin konseptin, pelin kulun, omat suunnittelupäätökset, rajauksen ja avoimet asiat. Tässä projektissa se täytetään Suunnitelma-näkymässä ja tallennetaan repositoryyn nimellä gdd.md.
GitHub-issue vko 44	*Yksi rajattu muutos peliin GitHubissa.* GitHub-issue kuvaa yhden rajatun muutoksen peliin: otsikko, perustelu, rajattu muutos ja Valmis kun -ehto. Tässä projektissa yhden issuen työmäärä on puolesta päivästä yhteen päivään. Sivun yksi työvaihe voi sisältää useita issueita.
backlog vko 44	*Priorisoitu issue-lista.* Backlog on projektin kaikkien GitHub-issueiden lista. Tässä projektissa se on GitHubin Issues-lista, jossa jokaisella issuella on prioriteetti: pakollinen (P0), tärkeä (P1) tai lisä (P2). Lista päivitetään aina, kun asiakas päättää jotain uutta.
mockup vko 44	*Käyttöliittymän luonnoskuva.* Mockup on paperille tai piirto-ohjelmalla tehty luonnos ruuduista ennen koodaamista. Tässä projektissa se on project-docs/evidence/week-44/mockup.png, ja ruudut rakennetaan sen mukaan.
UI vko 44	*User interface, käyttöliittymä.* Käyttöliittymä on se osa peliä, jonka pelaaja näkee ja jota hän käyttää: painikkeet, tekstit ja paneelit. Unityssä ne rakennetaan Canvas-alueelle.
feature vko 44	*Pelin yksittäinen ominaisuus.* Feature on yksi pelaajalle näkyvä ominaisuus, esimerkiksi pistelasku tai viiden parhaan tuloksen lista. Tässä projektissa featuret tehdään yksi kerrallaan, yleensä yksi viikossa.
scene vko 43	*Unityn näkymä.* Scene on Unityn tiedosto, jossa pelin objektit ovat. Tässä projektissa koko peli on yhdessä scenessä nimeltä CafeGame.
Hierarchy vko 45	*Unityn objektilista.* Hierarchy-ikkuna näyttää scenen kaikki objektit puuna. Siinä luodaan ja nimetään esimerkiksi Canvas, paneelit, painikkeet ja GameManager.
metodi vko 45	*Skriptin toiminto.* Metodi on C#-skriptin nimetty toiminto, esimerkiksi StartGame. Painike kutsuu metodia, kun pelaaja klikkaa sitä.
TODO vko 45	*Täydennettävä kohta koodissa.* TODO-rivi on työpohjaan merkitty kohta, jonka kirjoitat itse. Rivi alkaa merkeillä // TODO, ja sen perässä lukee, mitä kohtaan tulee.
hyväksymistesti vko 45	*Testi, joka osoittaa työn valmiiksi.* Hyväksymistesti kertoo, mitä tehdään ja mitä pitää näkyä, jotta työvaihe tai muutos on valmis. Esimerkiksi Aloita → Kahvi → Toimita → tulosruudulla 10.

Canvas vko 45	*Käyttöliittymän alue Unityssä.* Canvas on alue, jolle Unityssä sijoitetaan tekstit, painikkeet ja paneelit. Pelin kolme ruutua ovat Canvasin sisällä.
Inspector vko 45	*Unityn asetusikkuna.* Inspector näyttää valitun objektin asetukset ja skriptien kentät. Siihen raahataan esimerkiksi paneelit ja products.json-tiedosto.
asset vko 45	*Peliin tuotava valmis tiedosto.* Asset on peliin tuotava valmis kuva-, ääni- tai fonttitiedosto, esimerkiksi sprite eli hahmon tai esineen kuva. Kirjaa jokaisesta assetista lähde ja lisenssi.
JSON vko 44	*Tekstimuoto datalle.* JSON on yksinkertainen tekstimuoto, jossa tieto on nimi–arvo-pareina. Tässä projektissa kahvilan tuotteet ovat products.json-tiedostossa, joten valikoimaa voi muuttaa koskematta koodiin.
Console vko 46	*Unityn viesti-ikkuna.* Console näyttää skriptien Debug.Log- ja Debug.LogError-viestit sekä virheet. Avaa se valinnalla Window → General → Console.
rajatilanne vko 47	*Pelin ääritilanne.* Rajatilanne on kohta, jossa peli menee helpoimmin rikki, esimerkiksi aika 0, nopea kaksoispainallus tai tyhjä nimimerkki. Rajatilanteet testataan erikseen.
vertainen vko 48	*Toinen opiskelija.* Vertainen on toinen opiskelija, joka keskustelee kanssasi, testaa peliäsi tai lukee koodiasi. Hän ei ole projektin asiakas.
PlayerPrefs vko 49	*Unityn pieni tallennuspaikka.* PlayerPrefs tallentaa pieniä tietoja avain–arvo-pareina. Selainversiossa tieto säilyy selaimen muistissa, kunnes pelaaja tyhjentää sen.
katselmointi vko 50	*Työn yhteinen tarkastus.* Katselmoinnissa toinen ihminen kokeilee peliä tai lukee koodia ja antaa palautetta. Viikolla 50 asiakas katselmoi pelin, viikolla 4 ohjaaja tai vertainen katselmoi koodin.
branch vko 51	*Git-haara.* Branch eli haara on rinnakkainen kehityslinja: teet muutoksen omassa haarassa, jolloin toimiva main-haara pysyy ehjänä. Valmis haara yhdistetään eli mergetään takaisin main-haaraan.
pull request vko 51	*PR, pyyntö yhdistää haara pääversioon.* Pull request eli PR on GitHubissa tehtävä pyyntö yhdistää oma haara main-haaraan. Se antaa katselmoinnille oman paikan ennen yhdistämistä. Pienen muutoksen voi myös yhdistää suoraan ilman PR:ää.
T01 vko 3	*Testitapauksen tunnus.* Testitapaukset numeroidaan juoksevasti: T01 on ensimmäinen testitapaus, T02 toinen. Odotettu tulos kirjataan ennen ajoa, ja tunnuksella viitataan testiin päiväkirjassa ja näyttömatriisissa.
refaktorointi vko 4	*Koodin rakenteen selkeyttäminen.* Refaktoroinnissa koodia muutetaan selkeämmäksi niin, että peli toimii täsmälleen kuten ennen. Sama testitapaus ajetaan ennen ja jälkeen muutoksen.
regressiotesti vko 3	*Vanhan toiminnan uusintatesti.* Regressiotesti on toinen testitapaus, joka käyttää samaa koodia kuin korjaus. Kun se menee läpi, tiedät, ettei korjaus rikkonut muuta.
RC vko 5	*Release candidate, julkaisuehdokas.* Julkaisuehdokas on lähes valmis versio, joka testataan täsmälleen siinä muodossa, jossa se aiotaan julkaista. RC1 on ensimmäinen julkaisuehdokas, eikä siihen enää lisätä uusia ominaisuuksia.
tagi vko 5	*Versionhallintaan merkitty nimetty versio.* Tagi on yhteen committiin kiinnitetty nimilappu, esimerkiksi RC1 tai v1.0. Tagin avulla löydät myöhemmin täsmälleen sen version, jonka asiakas testasi.

Näyttömatriisi – 32 osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelvata useaan kohtaan.

Ohjelmointi

- □ **Kehitysympäristö** – Editorikuva, build ja repository.
- □ **Virheenkorjaus** – Kolme täydellistä ketjua aidoista havainnoista tai opettajan merkityistä vikatehtävistä.
- □ **Toimintojen testaus** – 12 testiä odotettuine ja toteutuneine tuloksineen.
- □ **Rakenteinen ohjelmointi** – Vastuutettu koodirakenne ja katselmointi.
- □ **Ylläpidettävä koodi** – Selkeät nimet, pienet metodit ja refaktorointicommit.
- □ **Käyttöliittymä** – Mockup sekä ennen/jälkeen-kuvat.
- □ **Toimintojen toteutus** – GitHub-issuet, valmis kun -ehdot ja commitit.
- □ **Tehtävistä sopiminen** – Tehtävälista ja kokousmuistio.
- □ **Ratkaisuvaihtoehdot** – Kahden toteutustavan vertailu ja päätös.
- □ **Ratkaisujen arviointi** – Koodi- tai peliversion katselmointiloki.
- □ **Oma toiminta tiimissä** – Itsearviointi ja seuraava kehitysaskel.

Ohjelmistokehittäjänä toimiminen

- □ **Asiakkaan tarpeet** – Kysymyslista ja aloitusmuistio.
- □ **Asiakaslähtöinen viestintä** – Ymmärrettävä yhteenveto vaihtoehdoista ja rajoista.
- □ **Version katselmointi** – Lokit viikoilta 50 ja 5.
- □ **Priorisointi** – Backlog P0-, P1- ja P2-tasoineen ennen ja jälkeen palautteen.
- □ **Tehtäviksi jakaminen** – 0,5–1 päivän issuet ja nimetyt tekijät.
- □ **Suunnittelu ja arviointi** – Työmääräarvio verrattuna toteumaan.
- □ **Toimintalogiikka** – Tilaus, toimitus, pisteet, aika, vaikeus ja pelitilat.
- □ **Tietovaraston valinta** – JSON- ja tallennusratkaisun perustelu.
- □ **Yhteys tietovarastoon** – Tuotedatan luku sekä tuloksen tallennus ja lataus.
- □ **Rajapinnat ja tieto** – Tiedonkulun kuvaus ja sitä vastaava koodi.
- □ **Tietoturva** – Syötteiden, tallennuksen ja salaisuuksien riskit sekä ratkaisut.
- □ **Versionhallinta** – Jatkuva Git-historia ja toimiva main.
- □ **Osan liittäminen** – Testattu yhdistäminen (merge) tai pull request.
- □ **Tuotantojulkaisu** – Toimiva v1.0-julkaisulinkki.

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

- □ **Kehittämisympäristön käyttöönotto** – Unity Hub, sovittu Unity-versio, 2D-projekti ja WebGL-testibuild.
- □ **Kirjaston mahdollisuudet ja rajoitteet** – Viikon 49 kirjaus tallennusratkaisun rajoituksista ja viikon 6 julkaisupäätökset tunnettuine puutteineen.
- □ **Kirjaston toiminnot ja työkalut** – Canvas-käyttöliittymä, Inspector-kytkennät, Play Mode ja Build Profiles committeina ja kuvina.
- □ **Ulkoiset komponentit** – TextMeshPro ja grafiikkapaketit tuotuina; lähteet ja lisenssit kirjattu GDD:hen tai README:hen.
- □ **Suunnittelu, toteutus ja testaus kirjastolla** – GDD:n omat päätökset ja backlog, featuret 1–7 committeina, testitapaukset T01–T12.
- □ **Julkaisu asiakkaan ympäristöön** – Unity WebGL -build GitHub Pagesissa; asiakas on kokeillut julkaistua linkkiä.
- □ **Ohjelmiston dokumentointi** – README, käyttöohje, gdd.md ja projektipäiväkirja project-docs-kansiossa.

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.